

ZLC 设备与实验手册

设备配置/加载 / 相机读出原理 / 标定与结果对象 / 实验流程

devices 层契约、camera capture/readout 教程
(sitemap/thresholds/detect)、TrapCalibration、虚拟后端与完整实验循环

2026-07-06

Contents

1	写给新读者	1
1.1	为什么单独一本设备与实验手册	1
1.2	三种后端, 同一套 notebook 接口	1
2	设备配置与加载	3
2.1	配置就是一个字典	3
2.2	load_devices 与 DeviceSet	3
2.3	设备注册表	4
2.4	探测总线: discover_devices	4
2.5	设备管理器 GUI: na.device_manager() / exp.device_manager()	4
2.6	存 / 取实验配置: save_config / load_config / open_devices	5
2.7	运行时控制 tab: runtime_controls()	6
2.8	换 / 停设备时, 引用它的逻辑节点会被停	7
2.9	设备契约 (四种类型)	7
3	相机采集原理	9
3.1	capture 返回什么	9
3.2	虚拟相机怎么造图	9
3.3	相机由脉冲门控: 没有脉冲就没有帧	10
4	相机读出教程: 从原始图到占据	11
4.1	第一步: sitemap (定位格点中心)	11
4.2	第二步: thresholds (学亮/暗边界)	12
4.3	第三步: detect (单张图判占据)	12
4.4	检测时间扫描 (保真度 vs 曝光)	13
5	标定与结果对象	14
5.1	TrapCalibration	14
5.2	逐站点读出保真度: 参考 bracket 标定	14
5.3	结果对象与 history	15
6	一次实验完整走一遍	16
6.1	典型 notebook 循环	16

6.2	脉冲、扫描与读出怎么衔接	16
7	离线虚拟后端	18
8	设备配置与加载 (深入)	19
8.1	设备层的文件地图	19
8.2	一份配置长什么样	19
8.2.1	最小 JSON 示例	20
8.2.2	等价的 Python dict 示例	20
8.2.3	设备之间的依赖: <code>\$device:name</code>	21
8.3	注册表: 短名到类的映射	21
8.3.1	<code>register_device_class</code> : 加入自定义设备	22
8.3.2	<code>available_device_configs</code> 与 <code>read_config</code>	22
8.4	<code>DeviceSet</code> : 一组设备的容器	22
8.4.1	三个类型化属性	22
8.4.2	字典式访问	23
8.4.3	<code>.require(name, expected_type)</code>	23
8.4.4	<code>.snapshot()</code>	23
8.5	打开与关闭的生命周期	23
8.5.1	<code>.open()</code> / <code>.close()</code> 的顺序规则	23
8.5.2	推荐的使用范式	24
8.6	设备契约: 每类设备必须提供什么	24
8.6.1	<code>BaseDevice</code> : 所有设备的公共契约	24
8.6.2	<code>CameraDevice</code> : 相机额外契约	24
8.6.3	<code>SequencerDevice</code> : 时序器额外契约	25
8.6.4	<code>TrapArrayDevice</code> : 阱阵列额外契约	25
8.7	五个时序器类: 选型决策表	26
8.7.1	选型速查	26
8.7.2	<code>RemoteSequencer</code> 的配置示例	26
8.8	编写并注册一个自定义设备	27
8.8.1	步骤一: 继承基类、实现契约方法	27
8.8.2	步骤二: 注册短名	28
8.8.3	步骤三: 在配置里使用	28
8.9	把整章串成一条最小工作流	29
9	相机采集原理 (深入)	30
9.1	相机抽象契约: <code>base.py</code> 中的 <code>CameraDevice</code>	30
9.1.1	属性与配置	30
9.1.2	采集: 设备原语与 <code>exp.capture</code> 的区别	30
9.1.3	核心不变量: <code>capture</code> 只显示原始图像	31
9.2	相机与时序器的协作: <code>arm-before-fire</code>	31
9.3	曝光即真值 (<code>exposure-as-truth</code>)	32

9.4	两种实现: qCMOS 真机 vs Virtual 仿真	32
9.4.1	QCMOSCamera (qcmos.py): 真实硬件	32
9.4.2	VirtualCamera (virtual.py): 仿真相机	33
9.4.3	仿真模型与参数 (脉冲驱动, 对标真实 Rb87 v16 qCMOS)	33
9.4.4	force_all_sites: 渲染满阵列 (仿真专属)	35
9.5	一个可运行的采集示例	35
9.6	第二只相机: MOT 监视相机与线圈磁场寻优	36
9.6.1	两种实现: PylonCamera 真机 vs VirtualMotCamera 仿真	36
9.6.2	自动发现与一键接线: discover_devices	36
9.6.3	脉冲模板: 三个 DAC api 槽就是三路线圈	37
9.6.4	MOT 光强: 一条单源规则	37
9.6.5	手动扫: Pulse scan + monitor_camera + grid facet	37
9.6.6	一键寻优: Optimize MOT field task	38
10	相机读出教程与原理 (深入)	39
10.1	总体框架: 标定一次, 探测无数次	39
10.2	第一步: site-map 标定 (找中心)	40
10.2.1	调用与返回	40
10.2.2	参数逐个解释	40
10.2.3	原理: find_site_centers 怎么找中心	41
10.2.4	位点排序: row-major / serpentine / column-major	41
10.3	第二步: thresholds 标定 (定亮暗分界)	42
10.3.1	调用与返回	42
10.3.2	原理: 从直方图到 Otsu 阈值	42
10.3.3	保真度: estimate_threshold_fidelity	43
10.4	进阶读出: PSF 加权提取 + 双高斯阈值 (Rb87)	43
10.4.1	PSF 加权提取: method="psf"	43
10.4.2	双高斯阈值: method="bimodal"	44
10.4.3	用实验数据逐站点定阈值 + 保真度: characterize_from_dir	44
10.4.4	逐站点网格: grid(sub_plot_kind=...)	45
10.5	第三步: detect 探测 (逐发判定)	46
10.5.1	调用与返回	46
10.5.2	原理: TrapCalibration.detect 是纯阈值二分类	46
10.5.3	一致性约束: reducer 与 radius 必须匹配	47
10.6	第四步 (可选): detection_time 扫描成像时间	47
10.6.1	调用与返回	47
10.6.2	原理: 用保真度曲线挑成像时间	47
10.7	保存、加载、清除与当前标定	47
10.8	一次完整的端到端示例	48
10.9	常见失败与排查	48

11	标定结果对象与完整实验流程（深入）	50
11.1	TrapCalibration 深入：每个字段的含义	50
11.1.1	字段清单	50
11.1.2	方法清单	51
11.1.3	schema 与存盘格式注意事项	51
11.2	结果对象与“覆盖层不改原图”不变量	52
11.2.1	结果对象家族	52
11.2.2	关键不变量：覆盖层永不改写原始图像	53
11.3	完整 notebook 实验流程	53
11.3.1	会话与子系统总览	53
11.3.2	configure_imaging 的四个参数	53
11.3.3	标准采集循环（可运行）	54
11.3.4	每一步产出了什么	54
11.4	扫描配合读出：一次扫描一帧一判定	55
11.5	排错清单	55
11.5.1	预检（preflight）失败	56
11.5.2	曝光错配（exposure mismatch）	56
11.5.3	换相机后标定过期（calibration staleness）	56
11.5.4	快速自检表	56
12	修改指南	58
12.1	我要加一台自定义设备	58
12.2	我要换标定的 reducer 或网格顺序	58
12.3	我要改本手册	58

1

写给新读者

1.1 为什么单独一本设备与实验手册

主手册讲系统总览，frontend 手册讲 GUI 与绘图，fpga 手册讲脉冲序列器硬件。但一台真实的中性原子实验还有一整层东西没讲清楚：**设备怎么配置和加载、相机怎么采图、原始图像怎么变成“每个格点有没有原子”、一次实验的完整流程长什么样**。这本手册专门补这一层，面向**第一次接手这套代码、想跑通一次成像 + 读出**的人。

devices/ 硬件适配器与设备契约。`base.py` (设备基类)、`registry.py` (配置加载)、`virtual.py` (离线模拟)、`qcmos.py` (真实相机)、`sequencer.py` (脉冲序列器客户端)。

core/ 纯算法与结果对象。`calibration.py` (`TrapCalibration`)、`analysis.py` (找格点、阈值、检测)、`results.py` (结果对象)。

subsystems/ 实验级能力束。`exp.readout` (读出)、`exp.timing` (时序)。

views/ 接到 frontend 的绘图适配器 (在原始图上叠格点圈)。

一个贯穿全书的不变式

`exp.capture()` 只显示**原始**相机图像，永远不画格点圈或阈值叠加。格点圈、占据判定属于**读出结果** (`sitemap/detect`)，不属于相机设备。如果你在一张 `capture` 上看到了格点圈，说明把标定状态错放进了相机层。

1.2 三种后端，同一套 notebook 接口

`na.connect` 返回一个 `NeutralAtomSession`，三种典型连法**暴露完全相同的接口**，只是底层设备不同：

Python

```
import Zou_lab_control.neutral_atom as na

# 1) 完全离线：虚拟相机 + 虚拟序列器 (教学、写代码、回归测试)
exp = na.connect("virtual")
```



Figure 1.1: 三种后端共享 `NeutralAtomSession` 接口。教学/回归用 `virtual`，真实实验用 `qCMOS` + 本地或远程序列器。

```
# 2) 真实硬件：控制电脑连 FPGA 电脑上的 server
exp = na.connect("remote_template",
                 sequencer={"host": "192.168.0.20", "port": 18861},
                 open_devices=True)
```

`session` 暴露四个子系统/设备入口：`exp.devices` (底层设备集)、`exp.camera` (采图)、`exp.timing` (脉冲/触发)、`exp.readout` (标定与占据检测)。

connect 怎么挑后端——没有“后端开关”。 `connect` 的第一个参数是一份配置：可以是内置名 (`"virtual"`)、一个 `.json` 路径、或一个 `dict`。后端**不是**一个单独的标志位——它就是配置里每个设备条目的 `type`：`DeviceSet` 按 `type` 分类设备 (`VirtualCamera` vs `QCMOSCamera`、`VirtualSequencer` vs `RemoteSequencer`)，**不看名字**。所以 `"virtual"` 只是“全虚拟配置”的快捷名；要上真机就换各条目的 `type` (这正是 `remote\_template.json` 已经替你做好的)。这就是“换真机只改 `connect()`”的机制根源：上层 GUI / 逻辑节点 / 读出代码只认 `NeutralAtomSession` 接口，对底下是哪种 `type` 一无所知。**唯一的额外差别**：虚拟后端自带 `VirtualTrapArray` (知道阱网格)，真机没有，所以真机上 `exp.readout.sitemap(...)` 要显式传 `grid\_shape=(ny, nx)`；其余调用完全相同。

2

设备配置与加载

2.1 配置就是一个字典

设备由一份配置描述，可以是内置名字（"virtual"）、一个 .json 路径、或一个 Python 字典。结构是**每个设备一条** `\{type, params\}`：

设备配置 JSON

```
{
  "trap_array": { "type": "VirtualTrapArray", "params": { "grid_shape":
    ↪ [5, 7] } },
  "camera":      { "type": "VirtualCamera",    "params": { "trap_array":
    ↪ "$device:trap_array" } },
  "sequencer":   { "type": "VirtualSequencer", "params": {} }
}
```

type 设备类名（内置短名，或完整 import 路径）。

params 构造参数。"`\$device:name`" 表示**引用另一台已加载设备**（如相机引用 trap_array）。加载器按依赖顺序实例化，并检测环。

2.2 load_devices 与 DeviceSet

`na.load_devices(config)` (`registry.py`) 解析配置、按依赖顺序实例化，返回一个 `DeviceSet` 容器：

Python

```
devices = na.load_devices("virtual", open_devices=False)
devices.camera      # -> CameraDevice (带类型契约校验)
devices.sequencer   # -> SequencerDevice
devices.trap_array  # -> TrapArrayDevice
devices.snapshot()  # -> {"camera": {...}, "sequencer": {...}, ...} JSON
    ↪ 安全自省
devices.open()      # 按依赖顺序打开（相机最后开），失败时逆序回滚
```


`DeviceSet` 既是属性访问(`.camera`)也是字典访问(`devices["camera"]`); `.require(name, expected\_type)` 会校验类型契约。

2.3 设备注册表

内置设备类在 `DEVICE_CLASSES(registry.py)`: `QCMOSCamera`、`VirtualCamera`、`VirtualTrapArray`、`RuntimeSequencer`、`RemoteSequencer`、`ManualSequencer`、`VerilogSequencer`。自定义设备用 `na.register_device_class(name, cls)` 注册后即可在配置里按名字引用。

Python

```
na.available_device_configs()  # 列出 configs/ 下可用配置名
na.register_device_class("MyCamera", "mypkg.devices:MyCamera")
```

2.4 探测总线: `discover_devices`

`na.discover_devices()` (`registry.py`) 主动**探测总线**——枚举在线相机 (DCAM / Pylon / VISA 等) 与序列器连接, 返回一份“发现清单”。它**不构造设备**、也不改会话; 只回答“这台机器上现在插着什么”。缺库或空总线时它**记一条 note 而非 raise** (与 `confocal` 同款容错), 所以在没有硬件的开发机上调用也安全。它是“我该往配置里写哪些 `type`”的侦察工具, 和 `load_devices` (据配置**构造**设备) 互补。

2.5 设备管理器 GUI: `na.device\_manager()` / `exp.device\_manager()`

把上面这些——配置里加载出的 `DeviceSet`、以及 `discover_devices` 的总线探测——摆到一个窗口里看, 就是**设备管理器**。它有**两个入口**, 对应“还没连”与“已经连上”两种时机:

`na.device\_manager(config)`——设备初始化入口 (还没有 session) 从 notebook 里**一开始就想用 GUI 建/编辑配置再连接硬件**时用它。它**返回管理器窗口**; 窗口起步是一个**纯配置编辑器** (空白, 或用 `config` 播种——模板名如 `"virtual"/"remote\_template"`、一个 JSON 路径、或一个 `config dict`, 与 `na.connect` 吃的形态一样)。窗口的绿色按钮写着 **Init devices**: 按下即用当前配置 `na.connect`, 成功后**这个窗口原地升级**成新会话的管理器 (Loaded 卡激活、按钮变成 Apply)。连出的 session **通过 `window.session` 交回 notebook**——Init 之前是 `None`, 成功后就是活 session。

`exp.device\_manager()`——会话绑定入口 (已经有 session) 已经 `exp = na.connect(...)` 之后, 用它打开**当前会话**的设备管理器 (监控台顶栏的 `Devices` 按钮打开的是**只读查看器**, 不是这个完整编辑器——见下节)。每会话单例, 重复调用复用同一窗口。

Python

```
import Zou_lab_control.neutral_atom as na

# 入口一: GUI 初始化设备。返回窗口, 编辑配置 + 按 "Init devices" 后取回 session:
mgr = na.device_manager("virtual")  # 或空 na.device_manager() 从零搭配置
# .....在窗口里编辑配置、按 "Init devices".....
exp = mgr.session                  # Init 前为 None, 成功后即活 session

# 入口二: 已有 session 时打开它的设备管理器 (每会话单例)
exp.device_manager()
```

两个入口，一套窗口；纯 connect 仍是一行捷径

不想要编辑器时，`exp = na.connect(config)` 依旧是一行连接的捷径；`na.device\_manager` 是它的**可视化初始化面**——想先在 GUI 里核对/微调配置再连时才用。两个入口打开的是**同一种窗口**：`na.device\_manager()` 在 Init 成功那一刻把升级后的窗口**登记成该 session 的单例管理器**，所以之后 `exp.device\_manager()` 复用的就是它。设备管理器**不是** `zf` 上的模块函数：它展示 / 编辑的正是 `neutral_atom` 设备层的 `DeviceSet`，而 `frontend` 对 `neutral_atom` 是**单向密封**的（前端不 import na），窗口工厂只能接 na 递进来的 `DeviceSet` 或 na 提供的 Init 回调——所以它由 na 拥有，而另外三个 GUI（task console / 脉冲编辑器 / 图查看器）能脱离会话独立跑，才是 `zf.*` 模块函数。

窗口**按设备域分区**（相机 / 序列器 / 阱阵列 / 未来的射频源……）——每个域一张卡，列出该域下每台设备的类型与状态。分区不是相机专属硬清单，而是**读注册表的设备域**（与各测量的设备下拉框同一个源）：注册一个新域，管理器自动多出一张卡，无需改面板代码。顶部的**Scan hardware**按钮就地调用 `discover_devices` 探测总线，把发现到的设备并成一张“Discovered”卡。

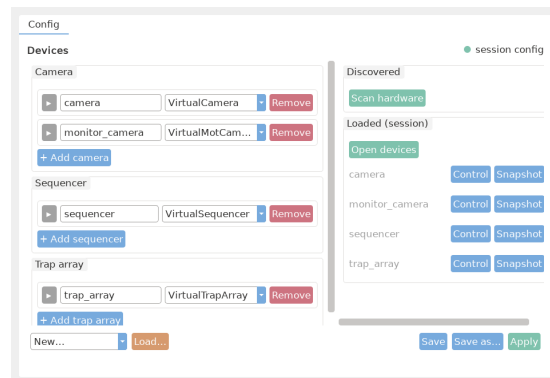


Figure 2.1: **设备管理器 GUI** (`exp.device\_manager()`): 把 `connect` / `load\_devices` 加载出的每个设备**按域分区**列出 (Camera / Sequencer / Trap array / 未来的 RF 源——与逐测量的设备下拉读的是**同一个注册表**)，底部 **Scan hardware** 按钮现场探一遍总线并把发现的设备补一张卡。顶部的**配置工具栏**——绿色 **Open devices** (初始化硬件)、**Load config** / **Save config** ——就是会话 `exp.open\_devices()` / `exp.load\_config()` / `exp.save\_config()` 的图形面 (存一份配置、下次一行连回来)。它是 `load\_devices` / `discover\_devices` 的**GUI 面孔**。两个入口: `exp.device\_manager()` (**会话绑定**, 编辑与换设备) 和 `na.device\_manager(config)` (还没有 session 时的**初始化**入口: 按 **Init devices** 即 `connect`, `window.session` 把新会话交回 notebook)。监控台顶栏的 **Devices** 按钮开的是**只读查看器** (`exp.device\_viewer()`，快照 + 运行时读回，**不编辑**、**不换设备**)，而非本编辑器——运行中改设备是危险的。本编辑器天生依赖会话的 `DeviceSet` (frontend 被密封、拿不到设备)，所以它**不是** `zf` 模块级函数；建 session 的模块级入口是 `na.device\_manager`。

2.6 存 / 取实验配置: save_config / load_config / open_devices

一份配置——就是 `na.connect(...)` 吃的那张 `\{角色: \{"type", "params"\}\}` JSON 表——往往是你现场**扫总线、填串号、调曝光试出来的**，值得存下来复用，而不是每次从头连一遍。设备管理器顶栏的**配置工具栏**和会话 API 是**同一套动作** (GUI 只是这三个方法的图形面)：

存 `exp.save\_config(path)` 把当前 `DeviceSet` 序列化 JSON 写到 `path` (缺 `.json` 后缀自动补，返回真正写出的 `Path`)。序列化走 `DeviceSet.to\_config()`——它是**唯一可回灌**

的序列化器 (返回配置的深拷贝, 与逐设备状态转储的 `snapshot()` 是两回事)。GUI 里点右侧 `Save config\ldots`。

取 `na.connect(path)` / `exp.load\_config(path)` `na.connect(path)` 从文件新开一个 session; 已有 session 时 `exp.load\_config(path)` 就把设备换成该配置——它先构造好新 `DeviceSet` 再原子替换 (坏配置不动老会话), 随后重建成像序列、清掉旧标定 (换了硬件, 旧 `sitemap/thresholds` 不再成立), 返回 `self`。GUI 里点 `Load config\ldots`, 设备列表随即刷新成新配置。

开 `exp.open\_devices()` 真正连接 / 初始化硬件 (等价于 `na.connect(..., open\_devices=True)`)。当你 `connect(..., open\_devices=False)` 只构造不打开、或刚 `load\_config` 换了一套设备时, 用它补上打开这一步 (相机总是最后打开)。GUI 里点左上角绿色 `Open devices`。

Python

```
exp = na.connect("virtual")                # 或某份 .json /
↳ 探测出来的配置
saved = exp.save_config("configs/my_experiment.json")  #
↳ 存下这次连接 (返回写出的 Path)

exp2 = na.connect("configs/my_experiment.json", open_devices=True)  #
↳ 下次一行连回来
# 或在已有 session 上就地换配置、再按需开硬件:
exp.load_config("configs/my_experiment.json")
exp.open_devices()
```

round-trip 契约

`save\_config` 写出的文件, `na.connect(那个文件)` / `load\_config` 必须重建出同一套 `DeviceSet`——一份调好的配置是可复用资产, 而不是每次重打。这条由契约测试 `tests/test\_device\_config\_io.py` 机械守卫。工具栏只在会话把这三个回调接进来时才出现 (`exp.device\_manager()` 会接); `show\_device\_manager(device\_set)` 裸开一个 `DeviceSet` 时没有可存取的话, 面板退化为只读 (设备列表 + Scan hardware)。

2.7 运行时控制 tab: `runtime\_controls()`

设备管理器的窗体是一个双 tab: **Config** tab 就是上面讲的配置编辑器 (写进 JSON 的构造期参数), **Control** tab 则是每台设备正在运行时可看可调的属性目录。二者是不同的层: Config 决定“用什么参数把设备造出来”, Control 决定“设备跑起来之后哪些旋钮还能现场拧”。Loaded 卡上每台设备行都有一个 **Control** 按钮, 点开就是这台设备的 Control 页 (一设备一 tab, 重开只聚焦不重复)。

这份目录由设备类的 `runtime_controls()` 声明 (`devices/base.py`): 每一项是一个 `RuntimeControl`, 绑定一个 `ParamDecl` (所以控件走与配置表单同一个 `PARAM_WIDGETS` 注册表按类型生成, 绝不 eval 自由文本) + 一个 `getter` (读回当前值) + 一个可选的 `setter` (写回; 没有 `setter` 的项是只读读回, 控件禁用)。Control 页每 200 ms 轮询 `getter` 刷新读回值; 可写项另有一个 Set 控件 + Apply 按钮, 点 Apply 把编辑值经设备自己的 `setter` 写回——值校验在设备边界做, 不在 GUI。

相机 (CameraDevice) 可写 `exposure` (经相机 `exposure` setter, 非正值被设备拒), 加只读读回 `roi` / `sensor` / `frame` (下一帧形状) / `trigger` (它数触发边沿的序列器线)。

序列器 (SequencerDevice) 只读读回 `firing` (此刻是否在连续发程序) / `last fired` / `scan` (扫描到第几点/第几轮, 或 idle) / `channels`; `sleep\_scale` (虚拟发间快进倍数) 仅当后端真带一个可写属性时才作为可写项出现——真实/远程序列器没有这个旋钮, 就退化成只读读回, 绝

不给一个写了会失败的 setter。

声明即控件，虚拟 == 真机

运行时控制目录由**契约基类**声明，所以每种后端（虚拟 / qCMOS / 远程）继承即得**同一套控件**——一台相机的 Control 页在虚拟和真机上长得一样，换真机不用改 GUI。某后端多一个 live 旋钮就在自己的 `runtime_controls()` 里**扩展**这份目录。所有项的 `decl.kind` 都在 `PARAM_WIDGETS` 覆盖内、值校验都在设备 setter——由契约测试 `tests/test\_device\_runtime\_controls.py` 守卫（`kind` ∈ 注册表、只读项无 setter、越界值被设备拒）。

2.8 换 / 停设备时，引用它的逻辑节点会被停

设备管理器的 `Load config / Apply`（换一套设备）与 `exp.close()`（拆整个会话）会**联动停止逻辑节点**——因为一个正在跑的监控台节点（相机 measurement、扫描 measurement、占据 processor.....）是在**后台线程里持有设备句柄**的消费者：设备一旦被换掉 / 关掉，这些线程若不先停，就会攥着旧句柄继续驱动一台已经不在的硬件。规则是**按设备身份精确停**，而不是无脑全停：

Load_config / device manager 的 Load•Apply——只停“引用了被换设备”的节点 会话先**构造** **好新 DeviceSet**（坏配置不动老会话），再按 role 比对新旧实例算出**被换掉的那些设备**，只停**引用了它们的运行节点**。所以**只换相机**时，一个跑在**没动的序列器**上的扫描**继续跑**——不误伤无关工作。

exp.close()——**停全部** 整个会话（连同监控台）都要拆，于是走**全量** teardown：停掉每一个节点，再关设备。

“节点引用了哪些设备”由 `LogicNode.referenced_devices()` 回答，它是**占用集的超集**：既收**独占驱动**（EXCLUSIVE，如相机 measurement 独占相机）、也收**只读观察**（OBSERVE，节点只读某设备状态）——两类都得停，因为无论驱动还是观察，设备没了这个节点就在对着空气跑。观察态设备被包成 `ReadOnlyDevice` 代理（与裸设备不是同一对象），判定“是否引用这台硬件”时先经 `underlying_device()` 反查真身，绝不因代理身份漏判。

为什么不“随便改设备”就行

不允许在有节点在跑时静默换掉它脚下的设备——那会让后台线程驱动死硬件。所以换设备是一个**显式的、会先停相关节点**的动作：换完设备后，被停的节点在监控台里回到“停”态，你确认新设备就绪再重新 Start。这条由契约测试 `tests/test\_device\_occupancy\_exclusion.py / test\_session\_device\_lifecycle.py` 守卫（换单设备只停引用它的、不动 disjoint 节点；close 停全部）。

2.9 设备契约（四种类型）

所有设备继承 `BaseDevice`（`open/close/stop/snapshot`）。三个**类型化**子类是强约束（用 `isinstance` 校验，不是鸭子类型）：

CameraDevice `exposure` 属性、`configure(exposure=...)`，以及三个采集原语：`arm(frames=None)`（就绪等外触发）、`read_frames(n) → list[np.ndarray]`（从设备自有缓冲阻塞取帧）、`disarm()`；`acquire(frames)` 是三者的免触发便捷组合。相机是纯取帧器：不碰序列器、不感知会话。

SequencerDevice `channels`、`clock_hz`、`prepare(sequence)`、`fire(sequence=None)`；可选 `wait_done(timeout)`、`abort()`、`set_safe_state()`。

TrapArrayDevice `n_sites`。刻意精简：它~~不~~暴露相机像素坐标的格点中心（那是标定的产物）。
`VirtualTrapArray` 额外有 `grid_shape/occupancy/render_image()/reload()`。

3

相机采集原理

3.1 capture 返回什么

`exp.capture(frames=1, camera=None, exposure=None, display=True)` 抓若干帧, 返回一个 `CaptureResult`。它是**会话级**编排 (相机本身不感知会话): 会话选定相机 (`camera` 指名任意一台, 缺省为读出主相机)、必要时写曝光并刷新成像序列, 然后走标准的 arm-before-fire 一发。

Python

```
res = exp.capture(frames=1, display=True)
res.images      # list[np.ndarray], 原始 uint16 帧
res.image       # 便捷取最后一帧
res.plot        # frontend 绘图对象 (只画原始像素, 无任何叠加)
```

`exp.capture` 内部调用测量层唯一的采集编排 helper `na.triggered_frames(camera, sequencer, sequence, frames)`: 先 `camera.arm(frames)` (返回时硬件已就绪等触发), 再 `sequencer.prepare(seq)`, `sequencer.fire(seq)`, 最后 `camera.read_frames(frames)` 从相机自有缓冲取回帧并 `disarm()`。相机是**纯 grabber**: 自己**不**驱动序列器、不数触发、不推断曝光; 序列器只流送脉冲。脉冲到达相机的唯一途径是**触发线**——真机是物理触发电缆, 虚拟相机在构造时接一根等价的“虚拟触发线” (设备配置里 “sequencer”: “\device:sequencer”)。

曝光时间的真相来源

曝光既能在相机上设 (`camera.configure(exposure=...)`), 也写在脉冲序列里 (probe 脉冲宽度)。约定: **序列里的曝光是真相来源**; 相机应当从序列取曝光, 而不是反过来。改曝光时优先改成像序列 (`exp.timing.configure_imaging(exposure=...)`)。

3.2 虚拟相机怎么造图

`VirtualCamera` (`virtual.py`) 用 `trap_array.render_image()` 合成图: 每个被占据的格点画一个高斯 PSF, 叠泊松噪声, 按曝光与原子寿命采样。它让你**不接硬件也能跑通整条读出链路**做开发和回归。

虚拟 vs 真实的一个差异

虚拟相机有一个仿真专属开关 `cam.force_all_sites = True` (设备属性, 默认 `False`): 打开后渲染器把每个格点都画成已装载原子, 方便确定性的设备级测试。生产路径从不使用它——sitemap 标定靠**对多帧真实** ~50% **装载求平均**找出全部站点, 与真机走同一条路。真实硬件没有这种捷径: 要“全亮”就必须真的装载原子。

3.3 相机由脉冲门控：没有脉冲就没有帧

相机是**外触发**器件：一帧的产生，只发生在脉冲序列器**正在发**一条带相机触发（emCCD）的连续脉冲、给出一个触发边沿的时候。这条因果链是**自底向上**建模的，虚拟与真实完全一致：

- **在跑脉冲 (On Pulse)**：序列器持续发 `repeat_forever` 的成像脉冲 -> 每个触发边沿采一帧 -> live 2D 图**持续刷新**。
- **停脉冲 (Stop Pulse / set_safe_state)**：序列器停发 -> 没有触发边沿 -> 相机**采不到新帧**。live 2D 图**冻结在最后一帧**，而不是凭空再造一张。

这条门控**只**存在于最底层的数据源里（“虚拟只 fake 最底层相机帧”这条铁律）：测量层把当前正在发的脉冲经 `acquire(sequence=...)` 交给相机，`VirtualCamera` 据此模拟它本该采到的帧（脉冲里没有相机触发边沿、或 `sequence=None` 就没有帧）；真实 `QCMOSCamera` 则忽略 `sequence`、直接等硬件触发边沿——没边沿就等不到帧。两者跑的是**同一条契约路径** (`camera.acquire(...)`)，只有“帧从哪来”不同。所以无论 you 从笔记本、task console 的 live 面板、还是真机看，“**脉冲在跑才有图、停脉冲就冻结**”这一行为都成立。

别让虚拟相机凭空造图

虚拟后端**绝不在**没有脉冲时也合成一张“理想成像”。那会让 live 2D 在停脉冲后仍然刷新，与真机背道而驰（真机此时相机什么也采不到）。一帧必须对应一个真实触发边沿——这正是“虚拟只 fake 最底层数据源、其余走实机同一路径”的体现：把相机的“有没有帧”也交给脉冲决定，而不是交给一个每拍都出图的假循环。

4

相机读出教程：从原始图到占据

读出分三步，每步产生一个**标定**并存进 session: **找格点 (sitemap)** -> **学阈值 (thresholds)** -> **单张检测 (detect)**。

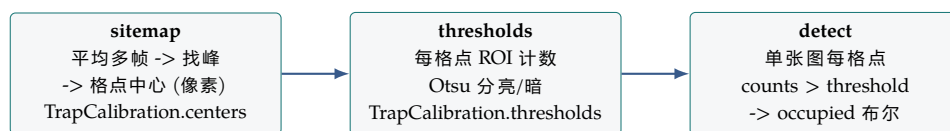


Figure 4.1: 读出三步: sitemap 定位、thresholds 学边界、detect 判占据。前两步是标定 (每次实验做一次), detect 是逐张测量。

4.1 第一步: sitemap (定位格点中心)

Python

```
sm = exp.readout.sitemap(frames=20, grid_shape=(5, 7),
    ↪ ordering="row-major",
                                roi_radius=1, reducer="mean", display=True)
sm.calibration.centers      # (35, 2) 每个格点的 (x, y) 像素坐标
sm.average_image           # 多帧平均图
sm.plot                    # 平均图 + 格点圈叠加
```

原理 (analysis.py 的 find_site_centers): 平均所有帧 -> 高斯平滑去噪 -> 局部极大值找峰 -> 按相对阈值筛掉背景 -> 按网格顺序 (row-major / serpentine / ...) 排序, 得到稳定的 (N_sites, 2) 像素中心。**sitemap 假设所有阱都被占据**, 它是标定不是测量。

4.2 第二步: thresholds (学亮/暗边界)

Python

```
exp.readout.sitemap(frames=6, display=False)          # 必须先有 sitemap
th = exp.readout.thresholds(frames=100, site=0, display=True)
th.calibration.thresholds    # (35,) 每个格点一个 Otsu 阈值
th.counts                    # (100, 35) 每帧每格点的 ROI 计数
th.fidelity                   # 所显示格点的判别保真度估计
```

原理: 对每帧每格点取 `roi_counts` (中心周围 $N \times N$ 像素盒, 按 `reducer` 归约成一个标量计数)。把 100 次的计数直方图化, 用 `otsu_threshold` 找**最大化类间方差**的阈值, 把“暗”(0-5 计数, 背景 + 读出噪声) 和“亮”(50-200 计数, 原子荧光) 分开。保真度 `estimate_threshold_fidelity` 用两个正态拟合亮/暗分布、算重叠积分得到。

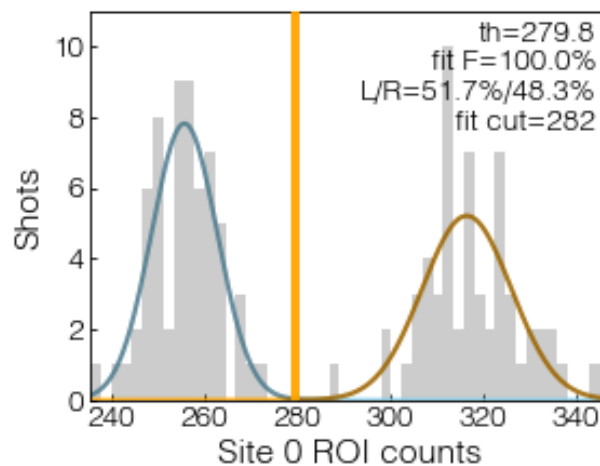


Figure 4.2: 虚拟后端**实跑**的 thresholds 标定直方图 (Site 0, 120 帧): 左峰为**暗态** (背景 + 读出噪声), 右峰为**亮态** (原子荧光), 并叠加亮/暗高斯拟合; 图中标注了 Otsu 阈值、拟合保真度与亮/暗占比。 `detect` 时把单张图每格点的 ROI 计数与该阈值逐位比较, 得占据布尔; 两峰分得越开、保真度越接近 1。本图由读出标定 `thresholds` 的 `plot_site` 直接产出, 而非示意图。

reducer 必须前后一致

阈值是**针对某个 reducer** (mean/sum/...) 学出来的。检测时若换了 reducer, 阈值就对不上。保持整次实验同一个 reducer (存在 `TrapCalibration.reducer` 里)。

4.3 第三步: detect (单张图判占据)

Python

```
det = exp.readout.detect(display=True, what="occupancy")
det.detection.occupied          # (35,) 布尔, 每格点有没有原子
det.detection.occupied_indices  # [被占据格点的下标]
det.detection.counts            # (35,) 该张图每格点的计数
det.plot                        # 原始图 + 被占据格点亮圈
```

原理 (`TrapCalibration.detect`): 取单张图每格点的读出信号 (box 为 ROI 计数), 逐元素

`counts > thresholds` 得布尔占据数组。这是[基于阈值的二元分类](#)，每张图独立判断，没有多帧贝叶斯/卡尔曼。

4.4 检测时间扫描（保真度 vs 曝光）

Python

```
scan = exp.readout.detection_time(times=[5e-6, 2e-5, 1e-4], shots=60,
                                   live=True, display=True)

scan.times          # (3,) 各检测曝光时间 (秒)
scan.fidelities     # 每个时间点的判别保真度
scan.stop()         # 提前停 (live 模式)
```

先在长曝光下采一组[参考](#)图建参考阈值，再对每个检测时间采 `shots` 张、算 Otsu 阈值与保真度。用它选一个“保真度已到平台”的成像时间。

5

标定与结果对象

5.1 TrapCalibration

读出三步的产物都封装在 `TrapCalibration` (`core/calibration.py`):

`centers` (`N, 2`) 格点中心像素坐标。

`thresholds` (`N,`) 每格点阈值 (或标量)。

`grid_shape` 阵列形状 (`ny, nx`)。

`roi_radius` ROI 半径 (计数用的像素盒大小)。

`reducer` 归约方式 `"mean"/"sum"/"median"/"max"`。

`metadata` 任意标志, 如 `\{"thresholds_calibrated": True\}`。

Python

```
path = exp.readout.save("calib.json")      # 存 .json (可读) 或 .npz (紧凑)
exp.readout.load(path)                    # 载回, 设到 exp 当前标定
exp.readout.current                        # 当前 TrapCalibration | None
exp.readout.clear()                       # 丢弃当前标定
cal = na.TrapCalibration.load("calib.json")
cal.detect(image)                         # 直接用标定检测一张图
```

5.2 逐站点读出保真度: 参考 bracket 标定

上面的 `thresholds` 用单批分布的 Otsu 给每个格点定一个阈值, 够日常占据判定。要**量化读出保真度**并在多种读出模型间挑最好的, 用参考 bracket 标定 (`CalibrateReadoutTask`, 控制台里 Add Panel 选 "Task: Calibrate readout", 或脚本里 `exp.readout.calibrate\_task(...)`)。它照搬单原子 Rb87 读出流程:

一发 = 一个 bracket 每发是一段**长-短-长**相机序列 (`reference\_bracket\_sequence`), 三帧成像**同一批原子** (阱全程开、触发之间不再冷却, 所以后一帧看到的是前一帧的幸存者)。两张长曝光参考帧**严格一致**时投票出该格点的真值占据; 不一致说明这一发原子丢了, 标为**歧义**并丢弃。

逐方法 held-out 保真度 三种读出模型——`box` (方形 ROI)、`psf` (逐站点匹配滤波)、`uniform\_psf` (共享一个核) ——各自在带标签的短读出帧上**训练**一个逐站点阈值, 再在**留出的**测试集上打分。因为三种模型对光子的加权不同, 在保真度受限 (短) 的读出下它们的保真度**各不相同** (匹配滤波通常优于方形盒) ——所以标定一次把三种都算出来, 下游的 `OccupancyProcessor` 选其一。

真值定边界 训练出的逐站点阈值**写回**标定 (`threshold\_method="per\_site\_reference"`), `detect` 据此判占据——边界来自真实标签, 而非单批 Otsu 速分。

一份标定因此能用**多种方法**读: 顶层 `box`, 其余在 `by\_method` 里; `calibration.signals(image, method=...)` 与 `thresholds\_for(method)` 按方法取, 且每种方法用**自己的背景模型** (PSF 减环形背景、`box` 不减), 保证阈值与信号在同一尺度上。标定跑完把每种读出方式的逐站点计数分布图与保真度写进一个可复看的文件夹 (`box/psf/uniform\_psf` 各一张)。

5.3 结果对象与 history

每个读出/采集操作返回一个**notebook 友好**的结果对象(`CaptureResult/SitemapResult/ThresholdResult/DetectionResult`) 都带一个 `.plot`, 并自动追加进 `exp.history`, 方便事后复盘与导出。

叠加是非破坏性的

格点圈是画在 matplotlib 坐标轴上的, **从不修改原始图像数组**。sitemap/detect 的 `.plot` 有圈, `capture()` 的没有——同一张原始图, 圈只是视觉层。

6

一次实验完整走一遍

6.1 典型 notebook 循环

Python

```
import Zou_lab_control.neutral_atom as na

# 0) 连接 (离线先用 virtual 跑通)
exp = na.connect("virtual")

# 1) 配成像时序 (曝光是真相来源)
exp.timing.configure_imaging(exposure=2e-3, load=True)
exp.timing.preflight().raise_if_failed()          # 跑前自检: 对齐/约束

# 2) 读出标定: 定位 -> 学阈值
exp.readout.sitemap(frames=20, grid_shape=(5, 7), display=True)
exp.readout.thresholds(frames=100, display=True)
exp.readout.save("calib.json")

# 3) 单张检测
det = exp.readout.detect(display=True)
print("loaded:", det.detection.occupied_indices)

# 4) 扫描测量 (live 画图)
scan = exp.readout.detection_time(times=[5e-6, 2e-5, 1e-4], shots=60,
    ↪ live=True)
```

6.2 脉冲、扫描与读出怎么衔接

`exp.timing` 拥有当前 `PulseSequence`, 并能 `bind_pulse(pulse)` 把一个 GUI 脉冲或 `PulseTableState` 绑到序列器做扫描 (详见主手册的 N-slot 扫描模型与 fpga 手册)。一次“扫描 + 采集”的闭环: 绑定脉冲与扫描表 -> 相机 arm -> 序列器 `prepare/fire` 逐扫描点无缝迭代、每点触发一次相机 -> 逐张 `detect` -> 把每点占据/计数汇成 1D/2D 结果画出来。

有限帧 vs 自由循环

真实相机采有限帧时，应让 readout/camera helper 先 arm 相机，再生成**刚好需要帧数**的有限触发序列；不要让 qCMOS 去等一个没有帧数边界的 `repeat_forever` 自由循环脉冲（那适合示波器找线，不适合定量采集）。

7

离线虚拟后端

`na.connect("virtual")` 给你一套完全离线的 `VirtualCamera + VirtualSequencer + VirtualTrapArray`。它对写实验逻辑、回归测试、教学非常有用：整条 `sitemap/thresholds/detect/scan` 链路都能跑，不接任何硬件。`VirtualTrapArray` 可设 `occupancy`、`reload()` 重新随机装载、`render_image()` 出图。本仓库的大量测试就跑在虚拟后端上。

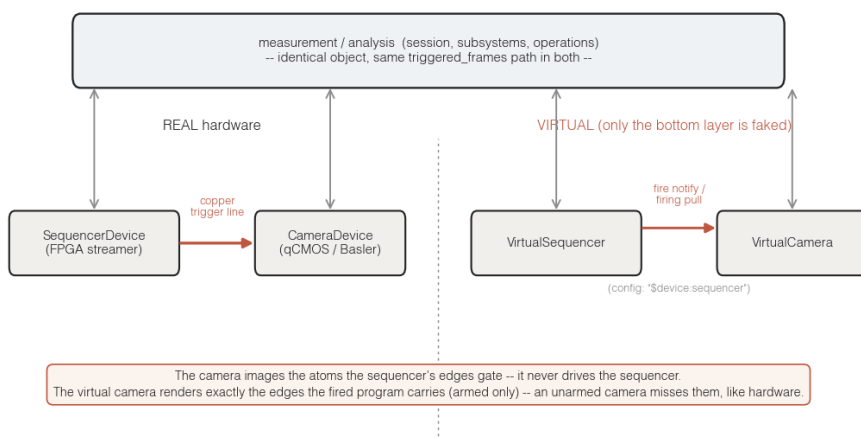


Figure 7.1: **虚拟 == 实机：只有最底层被仿真。** 上方的 measurement / analysis 栈 (session / subsystems / operations, 含唯一编排 `triggered\_frames`) 在真机与虚拟下是**同一个对象、走同一条代码路径**。真机里序列器用一根**铜触发线**接到相机；虚拟里 `VirtualSequencer` 的 fire 通知 / firing 拉取这根**被仿真的触发线**驱动 `VirtualCamera` (config 用 `"\device:sequencer"` 接线)。两边相机都**只按到达的触发边沿成像、从不驱动序列器**；虚拟相机严格按 fired 程序在其触发线上**真实携带**的边沿数渲染帧，**未武装就漏掉边沿**——和真机一模一样。所以换真机只改 `connect()`，上面一行分析代码都不用动。

8

设备配置与加载（深入）

本章是给“从零开始”理解本系统设备层的人写的一份参考手册。它回答四个核心问题：一份**设备配置**长什么样、`load_devices`与**DeviceSet**如何把配置变成可用的设备对象、所有设备共同遵守的**契约** (contract) 是什么、以及在五个时序器 (sequencer) 类之间如何取舍。每一节都尽量给出可直接运行的代码片段，并标注常见错误。

8.1 设备层的文件地图

设备层全部位于 `Zou_lab_control/neutral_atom/devices/` 目录下。先建立一张文件与职责的对照表，后面各节会反复引用。

devices/base.py 定义抽象基类：**BaseDevice** 是所有设备的根，三个领域基类 **CameraDevice**、**SequencerDevice**、**TrapArrayDevice** 在其上扩展各自的领域方法。它们规定了“一个设备必须提供哪些方法”，即设备契约。

devices/registry.py 注册表与加载器。包含 `load_devices`、**DeviceSet**、**DEVICE_CLASSES**、`register_device_class`、`available_device_configs`、`read_config`。这是你与设备层打交道的主入口。

devices/virtual.py 离线模拟实现：**VirtualCamera**、**VirtualTrapArray**、**VirtualSequencer**。没有任何真实硬件时用它们跑通整条流程。

devices/qcmos.py **QCMOSCamera**，真实的 qCMOS 相机驱动封装。

devices/sequencer.py 几个真实/半真实时序器实现(**RuntimeSequencer**、**RemoteSequencer**、**ManualSequencer**、**VerilogSequencer**) 的设备层包装。

心智模型

配置（一段 JSON 或 dict）是**描述**，`load_devices` 是**工厂**，**DeviceSet** 是**已经造好、可以打开使用的一组设备**。三者层层递进：先写描述，再实例化，最后打开。

8.2 一份配置长什么样

配置可以是三种形态之一，`read_config`与`load_devices`都接受它们：

1. 字符串 `"virtual"`: 一个内置快捷方式, 等价于加载内置的全虚拟配置 (虚拟相机 + 虚拟时序器 + 虚拟阱阵列), 用于无硬件冒烟测试。
2. 一个 `.json` 文件路径: 磁盘上的配置文件。
3. 一个 Python `dict`: 在代码里直接构造的配置, 跳过磁盘。

无论哪种形态, 最终的数据结构是统一的: 顶层是一个字典, 键是**设备名** (`device_name`, 你自己取的逻辑名字, 例如 `"cam"`、`"seq"`), 值是一个描述该设备的小字典, 含两个字段:

"type" 设备类的**短名** (ClassName), 例如 `"VirtualCamera"`。短名通过 `DEVICE_CLASSES` 映射到真正的类。

"params" 传给该类构造函数的参数字典。可以为空 `\{\}`。

8.2.1 最小 JSON 示例

下面是一份完整、自治的虚拟配置。把它存成 `my_virtual.json` 即可被 `load_devices` 加载:

Python

```
{
  "trap": {
    "type": "VirtualTrapArray",
    "params": { "n_sites": 16 }
  },
  "seq": {
    "type": "VirtualSequencer",
    "params": { "channels": 62, "clock_hz": 50000000 }
  },
  "cam": {
    "type": "VirtualCamera",
    "params": { "exposure": 0.01 }
  }
}
```

注意三个键 `trap/seq/cam` 是**你取的名字**, 不是固定关键字; `DeviceSet` 会根据每个设备的**类型** (是相机/时序器/阱阵列) 自动把它归类到 `.camera/.sequencer/.trap\_array`, 而不是看名字。

8.2.2 等价的 Python dict 示例

完全相同的配置在代码里这样写, 可直接喂给 `load_devices`:

Python

```
config = {
  "trap": {"type": "VirtualTrapArray", "params": {"n_sites": 16}},
  "seq": {"type": "VirtualSequencer", "params": {"channels": 62,
                                                  "clock_hz":
                                                    ↪ 50_000_000}},
  "cam": {"type": "VirtualCamera", "params": {"exposure": 0.01}},
}

from Zou_lab_control.neutral_atom.devices.registry import load_devices
```

```
devices = load_devices(config)  # 返回一个 DeviceSet
```

8.2.3 设备之间的依赖：\\${device}:name

`params` 里的某个值如果写成字符串 `"\${device}:name"`，表示“把另一个已经构造好的设备对象注入到这里”。这是设备之间互相引用的唯一机制。典型用途是让相机持有时序器的引用，或让某个组合设备依赖底层设备。

Python

```
config = {
    "seq": {"type": "VirtualSequencer", "params": {"channels": 62}},
    "cam": {
        "type": "VirtualCamera",
        "params": {
            "exposure": 0.01,
            "sequencer": "\${device}:seq"  # 注入上面那个 seq 设备对象
        }
    },
}
```

加载顺序与循环检测：`load_devices` 会分析这些 `\${device}:` 引用，按依赖顺序逐个实例化——被依赖者先建好，引用者后建。如果出现 A 依赖 B、B 又依赖 A 这样的**循环依赖**，`load_devices` 会检测出来并报错，而不是无限递归。

常见错误

- 把 `"type"` 写成全限定类路径而不是短名——除非你已经用 `register_device_class` 注册过该路径，否则应使用 `DEVICE_CLASSES` 里的短名（见下节）。
- `\${device}:name` 里的 `name` 拼错或指向一个不存在的设备键——加载会失败。
- 忘了某个 `params` 字段是设备引用，直接传了字符串字面量——只有以 `\${device}:` 为前缀的字符串才会被解析为引用。

8.3 注册表：短名到类的映射

`DEVICE_CLASSES` 是一张把**短名**映射到**实现类**的表。配置里 `"type"` 字段写的就是这些短名。内置短名包括：

QCMOSCamera 真实 qCMOS 相机 (`devices/qcmos.py`)。

VirtualCamera 离线模拟相机。

VirtualTrapArray 离线模拟阱阵列。

RuntimeSequencer 本地进程内时序器，自己编译边沿表。

RemoteSequencer 通过 RPyC 连到 FPGA 主机上 `SequencerService` 的客户端代理。

ManualSequencer 手动时序器，提示“现在手动启动 FPGA”，`fire()` 是占位空操作，用于首光调试。

VerilogSequencer 不执行而是生成 Verilog 的时序器，需要一个 `fire\_callback`。

8.3.1 register_device_class: 加入自定义设备

要把自己写的设备类接入配置系统，调用 `register_device_class(name, cls_or_path)`:

name 你要注册的短名字字符串，之后就能在配置的 `"type"` 里使用。

cls_or_path 既可以直接传类对象，也可以传一个可导入的字符串路径（让注册表在需要时再导入，避免提前加载重量级依赖）。

Python

```
from Zou_lab_control.neutral_atom.devices.registry import
↳ register_device_class

# 形式一：直接传类对象
register_device_class("MyCamera", MyCamera)

# 形式二：传可导入路径（延迟导入）
register_device_class("MyCamera", "my_pkg.cameras:MyCamera")
```

注册后，`"type": "MyCamera"` 就能在任何配置里使用。

8.3.2 available_device_configs 与 read_config

`available_device_configs()` 列出 `neutral_atom/configs/` 目录下随仓库附带的配置文件，外加内置字符串快捷方式 `"virtual"`，目前包括 `"virtual"`、`manual_template.json`、`remote_template.json`、`basler_monitor.json`。它返回这些可用配置，方便你在不记路径的情况下选用模板。注意 `"virtual"` 是**程序内置**的全虚拟配置（直接由 `VirtualTrapArray` 等设备类的默认参数构造），不是磁盘上的 JSON 文件；`load_devices("virtual")` 永远走这条内置路径。

`read_config` 负责把“三种形态之一”的输入**归一化**成那个统一的顶层字典：传 `"virtual"` 时返回内置虚拟配置，传路径时读盘解析 JSON，传 dict 时原样（校验后）返回。`load_devices` 内部就先经过它。

Python

```
from Zou_lab_control.neutral_atom.devices.registry import (
    available_device_configs, read_config, load_devices,
)

print(available_device_configs())          # 看有哪些模板
cfg = read_config("remote_template.json")  # 归一化成统一 dict
devices = load_devices(cfg)                # 实例化为 DeviceSet
```

8.4 DeviceSet: 一组设备的容器

`load_devices` 返回的不是裸字典，而是 `DeviceSet`——一个既能按名字访问、又对三类核心设备做了**类型化**、**契约校验**的容器。

8.4.1 三个类型化属性

.camera 该组里那个相机设备（必须是 `CameraDevice` 的实例）。

.sequencer 时序器设备（`SequencerDevice`）。

`.trap\_array` 阱阵列设备 (`TrapArrayDevice`)。

这三个属性在访问时做契约检查：如果配置里某个被归为相机的设备并不满足 `CameraDevice` 契约，就会暴露出来，而不是等到运行实验时才神秘失败。

8.4.2 字典式访问

`DeviceSet` 同时支持按名字访问，行为像字典又像命名空间：

`ds["seq"]` 用 `\_\_getitem\_\_` 按设备名取设备。

`ds.seq` 用 `\_\_getattr\_\_` 用属性语法取同一个设备（名字合法时）。

`"seq" in ds` 用 `\_\_contains\_\_` 判断某名字是否存在。

8.4.3 .require(name, expected_type)

当你需要“既按名字取、又强制类型正确”时用 `require`。它取出名为 `name` 的设备，并断言它是 `expected\_type`（或其子类）；不满足就报错。这比裸 `ds[name]` 更安全，适合在实验脚本入口处做前置校验。

Python

```
from Zou_lab_control.neutral_atom.devices.base import SequencerDevice

seq = devices.require("seq", SequencerDevice)    # 取出并强制类型
print("seq" in devices)                          # True
print(devices["cam"] is devices.camera)          # 多半为 True
```

8.4.4 .snapshot()

`ds.snapshot()` 遍历组内每个设备、调用各自的 `snapshot()`，汇总成 `\{name: device.snapshot()\}` 形式的字典，并且保证是 **JSON-safe**（可以直接 `json.dumps`）。这用于记录“本次实验用的设备状态/参数”，方便存档与复现。

Python

```
import json
state = devices.snapshot()
print(json.dumps(state, indent=2))    # 可序列化的设备状态全景
```

8.5 打开与关闭的生命周期

`DeviceSet` 把“打开全部设备”和“关闭全部设备”封装成两个方法，并规定了严格的顺序与失败回滚。

8.5.1 .open() / .close() 的顺序规则

相机最后打开、最先关闭 `open()` 时相机放在**最后**打开；`close()` 时相机**最先**关闭。原因是相机通常依赖时序器/阱阵列已经就绪（例如曝光要与时序对齐），所以让它在最稳定的窗口内开启、在拆解一开始就退出。

失败回滚 如果 `open()` 在中途某个设备上失败，`DeviceSet` 会**回滚**已经打开的那些设备（把它们关掉），而不是留下一半打开、一半未开的不一致状态。

8.5.2 推荐的使用范式

把 `open()/close()` 配对使用，确保无论实验成功还是抛异常都能干净关闭：

Python

```
from Zou_lab_control.neutral_atom.operations.measurement import
↳ triggered_frames

devices = load_devices("virtual")
devices.open()           # 按顺序打开，相机最后；失败自动回滚
try:
    # 唯一的 arm-before-fire 编排入口：先 arm 相机（返回即就绪），再 fire 序列器，
    # 最后从相机缓冲取帧。相机绝不漏触发；序列器只流送脉冲，不驱动相机。
    frames = triggered_frames(devices.camera, devices.sequencer, sequence,
↳ 1)
    frame = frames[-1]
finally:
    devices.close()       # 相机最先关，逆序拆解
```

常见错误

- 自己手动逐个 `open()` 设备而不走 `DeviceSet.open()`——会丢掉顺序保证和失败回滚。
- 实验抛异常后忘记 `close()`，导致相机/硬件句柄泄漏。务必用 `try/finally`。
- 在 `open()` 之前就调用 `acquire()/fire()`——设备尚未就绪。

8.6 设备契约：每类设备必须提供什么

契约由 `devices/base.py` 的基类定义。理解契约就能在不看具体实现的情况下知道“对一个设备能调什么”。

8.6.1 BaseDevice: 所有设备的公共契约

open() 打开/初始化设备，进入可用状态。

close() 关闭设备，释放资源。

stop() 停止当前活动（区别于 `close`：停止动作但不一定释放句柄）。

snapshot() 返回该设备当前状态的 JSON-safe 字典，供 `DeviceSet.snapshot()` 汇总。

8.6.2 CameraDevice: 相机额外契约

在 `BaseDevice` 之上增加：

exposure 曝光时间（属性/配置项）。

configure(...) 配置相机参数。

arm(frames=None) 就绪等外触发；返回时硬件已武装（fire 永远发生在 arm 返回之后——这就是 arm-before-fire 的时序保证）。`None` 表示连续采集。

read_frames(n) 从设备自有缓冲阻塞取 `n` 帧。武装期间到达的帧全部无损排队（防丢帧缓冲），先 fire 后取也一帧不丢。

disarm() 解除武装。

acquire(frames) 三原语的便捷组合 (arm + read + disarm)，适合免外触发的自由运行场景。

相机对更大系统唯一“知道”的事实是 `capture\_trigger\_channels`——自己的触发输入接在序列器哪条线上——一个只向上暴露、从不用来控制序列器的被动布线事实。

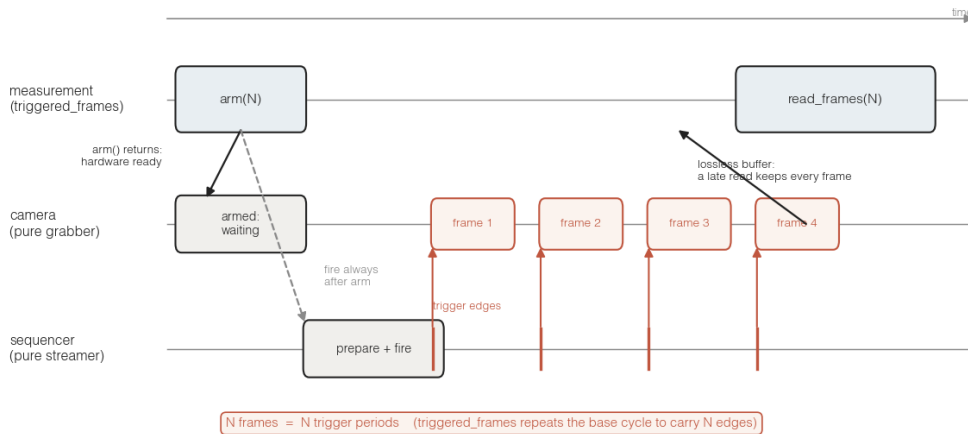


Figure 8.1: 纯 grabber 的 arm-before-fire 时序 (三泳道: measurement / camera / sequencer, 同一条时间轴)。相机先 `arm(N)`，返回时硬件已就绪、等外触发——所以 fire 永远发生在 arm 返回之后，第一个触发边沿绝不丢失。随后 measurement 层的 `triggered\_frames` 让序列器 `prepare + fire` 一段携带 `N` 个触发边沿的程序 (把成像基周期重复到 `N` 个边沿)：每个边沿门控一帧进相机自有的无损缓冲，`read\_frames(N)` 再把它们取走——即使先 fire、后取帧，缓冲也一帧不丢。一句话记住：`N 帧 = N 个触发周期`；相机从不驱动序列器，只按到达的边沿取帧（虚拟后端亦然，只是把这根触发线也仿真了）。

8.6.3 SequencerDevice: 时序器额外契约

channels 通道数。

clock_hz 时钟频率 (Hz)，例如 `50\_000\_000` (50 MHz，对应 20 ns 一个 tick)。

prepare(program) 把编译好的序列/边沿准备进设备（真实硬件上即写入寄存器/内存）。

fire() 触发一次序列执行。

wait_done() 阻塞等待序列跑完（部分实现提供）。

abort() 中止当前执行（部分实现提供）。

set_safe_state() 让输出回到安全/空闲电平（部分实现提供）。

8.6.4 TrapArrayDevice: 阱阵列额外契约

n_sites 阱点 (site) 数量。

关键设计取舍

TrapArrayDevice 故意不暴露每个阱点在相机像素上的中心坐标。原因是：像素中心属于标定 (calibration) 结果，会随光路、相机位置而变，不应硬编码在设备里。阱阵列只负责“有多少个阱点”

这种内禀属性，像素映射由标定模块单独提供。这条边界划分能避免设备层和标定层耦合。

8.7 五个时序器类：选型决策表

时序器是本系统最容易混淆的部分，因为有五个实现，外形相似但用途迥异。下面逐一说明各自的**独特角色**与适用场景。

VirtualSequencer 离线 mock。prepare/fire 都是平凡 (trivial) 的空跑实现，不接触任何硬件。

何时用：写脚本、跑单测、调 GUI、CI——任何没有也不需要硬件的场合。

RuntimeSequencer 本地进程内运行一个 SequencerService，会真正**编译边沿表**。**何时用**：在控制电脑本地就能完成编译与（模拟/本地）执行，不经过网络的场合；也适合验证编译产物。

ManualSequencer 打印“现在手动启动 FPGA”，fire() 是**占位空操作**。**何时用**：**首光调试** (first-light commissioning) ——硬件还没接好自动触发链路，你想人工按下 FPGA 启动、用本系统准备好一切其余流程。

RemoteSequencer 通过 RPyC 连到 FPGA 电脑上运行的 SequencerService 的客户端代理。参数有 host/port/channels/clock_hz/trigger_channels。open() 时会从远端 snapshot **同步**通道数与时钟；prepare 把序列以 JSON 发过去，fire 调用远端执行。**何时用**：正式实验——控制电脑与 FPGA 电脑分离，跨网络驱动真实硬件。

VerilogSequencer 不执行，而是**生成 Verilog**。需要一个 fire_callback。**何时用**：你想把序列固化成 HDL、做综合/上板前的离线生成，而不是即时驱动。

8.7.1 选型速查

- 没有硬件、只想跑通流程 → VirtualSequencer (或整套 “virtual” 配置)。
- 本地编译、不过网络 → RuntimeSequencer。
- 硬件刚接好、需人工启动 → ManualSequencer。
- 正式跨机驱动 FPGA → RemoteSequencer。
- 生成 HDL 而非执行 → VerilogSequencer。

8.7.2 RemoteSequencer 的配置示例

正式实验里最常用的就是远程时序器。一份对应配置大致如下：

Python

```
config = {
    "seq": {
        "type": "RemoteSequencer",
        "params": {
            "host": "192.168.1.50",
            "port": 18861,
            "channels": 62,
            "clock_hz": 50_000_000
        }
    },
    "cam": {"type": "QCMOSCamera", "params": {"exposure": 0.005}},
}
```

```

    "trap": {"type": "VirtualTrapArray", "params": {"n_sites": 100}},
}

from Zou_lab_control.neutral_atom.operations.measurement import
↳ triggered_frames

devices = load_devices(config)
devices.open()      # RemoteSequencer.open() 会从远端 snapshot 同步
↳ channels/clock_hz
seqr = devices.sequencer
try:
    # 唯一的 arm-before-fire 编排入口: 相机先 arm (返回即就绪等触发), 随后 fire 序列器,
    # 相机绝不漏掉触发。序列器只流送脉冲, 不驱动相机; 真相机的帧来自传感器。
    frames = triggered_frames(devices.camera, seqr, program, 1)
    img = frames[-1]
finally:
    devices.close()

```

关于 `open()` 同步

对 `RemoteSequencer` 来说, 配置里写的 `channels/clock\_hz` 是期望值; `open()` 真正连上后会以远端 snapshot 为准把它们同步过来。所以如果本地配置和远端固件不一致, 以打开后的实际值为准——这能防止本地误填的参数悄悄带偏后续编译。

8.8 编写并注册一个自定义设备

把前面所有概念串起来: 要接入一个新设备, 你需要 (1) 继承正确的领域基类以满足契约, (2) 用 `register_device_class` 注册短名, (3) 在配置里用该短名引用它。

8.8.1 步骤一: 继承基类、实现契约方法

下面写一个最小的自定义相机。继承 `CameraDevice` 意味着你承诺实现相机契约里的方法 (这里给出关键几个):

Python

```

from Zou_lab_control.neutral_atom.devices.base import CameraDevice

class MyCamera(CameraDevice):
    def __init__(self, exposure=0.01, **params):
        self._exposure = exposure
        self._open = False

    # --- BaseDevice 契约 ---
    def open(self):
        self._open = True      # 这里换成真实的硬件初始化
    def close(self):
        self._open = False
    def stop(self):
        pass
    def snapshot(self):
        # 必须是 JSON-safe: 只放基本类型

```



```

        return {"type": "MyCamera", "exposure": self._exposure,
                "open": self._open}

# --- CameraDevice 契约 ---
@property
def exposure(self):
    return self._exposure
def configure(self, **kwargs):
    self._exposure = kwargs.get("exposure", self._exposure)

# 采集面 (arm/read_frames/disarm/acquire) 由基类实现; 子类只填三个硬件钩子:
def _arm(self, frames):
    ...                                # 让硬件进入外触发等待 (如 buf_alloc +
    ↪ cap_start)
def _grab(self, n, *, timeout=None, stop=None):
    # 从驱动取回一帧并交给基类缓冲 (_deliver), 返回 True; 超时无帧返回 False
    frame = ...                        # 等待硬件触发边沿产出的一帧
    self._deliver([frame])
    return True
def _disarm(self):
    ...                                # 停止采集 (如 cap_stop)

```

8.8.2 步骤二: 注册短名

Python

```

from Zou_lab_control.neutral_atom.devices.registry import
    ↪ register_device_class
register_device_class("MyCamera", MyCamera)

```

8.8.3 步骤三: 在配置里使用

Python

```

config = {
    "seq": {"type": "VirtualSequencer", "params": {"channels": 62}},
    "cam": {"type": "MyCamera", "params": {"exposure": 0.02}},
}
devices = load_devices(config)
devices.open()
try:
    assert devices.camera.exposure == 0.02
    print(devices.snapshot())    # 你的 snapshot() 会被汇总进去
finally:
    devices.close()

```

自定义设备的常见坑

- `snapshot()` 返回了非 JSON-safe 的对象 (如 numpy 数组、自定义类实例), 会让 `DeviceSet.snapshot()` 后续的 `json.dumps` 失败。只放基本类型 (数、字符串、列表、字典)。

- 继承了 `CameraDevice` 却没实现某个契约方法，访问 `DeviceSet.camera` 时的契约校验会暴露问题——这是好事，按提示补齐即可。
- `open()` 里发生异常时没有让设备保持一致状态——记住 `DeviceSet.open()` 会回滚已开设备，你的 `close()` 应当能安全地关闭一个“只打开了一半”的设备。

8.9 把整章串成一条最小工作流

最后给出一段从配置到采集、不依赖任何真实硬件、可立即运行的完整示例，作为本章的收尾参照：

Python

```
from Zou_lab_control.neutral_atom.devices.registry import (
    load_devices, available_device_configs,
)
from Zou_lab_control.neutral_atom.devices.base import (
    CameraDevice, SequencerDevice, TrapArrayDevice,
)

print(available_device_configs())      # 看看有哪些现成模板

devices = load_devices("virtual")      # 全虚拟：相机 + 时序器 + 阱阵列

# 类型化访问 + 契约校验
cam = devices.require("cam", CameraDevice) if "cam" in devices else
↳ devices.camera
seq = devices.sequencer
trap = devices.trap_array
print("sites:", trap.n_sites, "channels:", seq.channels, "clock:",
↳ seq.clock_hz)

devices.open()                        # 相机最后开；失败回滚
try:
    # 真正采集走唯一的 arm-before-fire 编排 helper (arm 相机 -> fire 序列器 -> 取帧)
    ↳ :
    # from Zou_lab_control.neutral_atom.operations.measurement import
    ↳ triggered_frames
    # frame = triggered_frames(cam, seq, sequence, 1)[-1]
    pass
finally:
    devices.close()                  # 相机最先关

import json
print(json.dumps(devices.snapshot(), indent=2))  # JSON-safe 状态全景
```

读到这里，你应该能够：写出三种形态的配置、理解 `\$device`：依赖与循环检测、用 `load_devices` 得到 `DeviceSet`、通过类型化属性与 `require` 安全取设备、遵守 `open/close` 的顺序与回滚、读懂四类设备的契约，并在五个时序器之间正确选型，乃至注册并接入自己的设备。

9

相机采集原理（深入）

本章自底向上讲清楚“相机层”到底做什么、不做什么。相机层是整个读出（readout）流水线的最前端：它的唯一职责是**把光子变成原始像素**，并在需要时与时序器（sequencer）协作完成外触发曝光。所有“原子在哪里、亮度够不够、是否装载”的判断都**不属于**相机层，而属于后续的 sitemap/detect 读出阶段。理解这条边界，是正确使用本系统、避免把标定状态错误地塞进相机里的关键。

9.1 相机抽象契约：base.py 中的 CameraDevice

所有相机（真实的 qCMOS 与仿真的 VirtualCamera）都实现同一个抽象基类 `CameraDevice`。这个契约定义了上层代码可以依赖的全部行为，使得“换硬件”对读出流水线透明。

9.1.1 属性与配置

exposure (property) 当前曝光时间。注意它只是相机当前持有的一个数值；**真正的曝光真值来自时序序列**（见后文“曝光即真值”一节），相机上的这个属性应当被序列驱动，而不是反过来去驱动序列。

configure(exposure=..., **kwargs) 配置相机参数。`exposure` 是最常用的关键字参数，用来设置曝光时间；`**kwargs` 允许具体实现接受额外的设备相关参数（例如读出速度、ROI 等）。该方法只改变相机内部状态，不触发任何采集。

相机**不感知实验会话**：它不持有 `exp`、不读时序。所有跨设备协作（成像序列、历史记录、显示）都在会话/测量层完成，相机只暴露上面的设备契约。

9.1.2 采集：设备原语与 `exp.capture` 的区别

系统刻意提供了两层采集入口，职责不同，初学者必须分清。

设备原语 `arm / read\_frames / disarm`（及组合 `acquire`） 底层采集，返回**原始帧列表**（每帧一个 `uint16 NumPy` 数组，未做任何处理）。`arm(frames)` 返回时硬件已就绪等触发；`read_frames(n)` 从设备自有缓冲阻塞取帧——武装期间到达的帧**全部无损排队**，先 fire 后取也一帧不丢；`disarm()` 收尾。相机是纯取帧器，绝不驱动时序器；需要外触发的一发由测量层唯一的编排 helper `triggered_frames(camera, sequencer, sequence, frames)` 完成

(arm → fire → read)。免触发的自由运行传感器直接 `acquire(frames)`。

exp.capture(frames=1, camera=None, exposure=None, display=True) → CaptureResult

面向交互/调试的**会话级**包装。会话选定相机、必要时写曝光并刷新成像序列,经 `triggered_frames` 拿帧后打包成 `CaptureResult` 并记入历史。

`CaptureResult` 上有几个常用字段:

.images 本次采集得到的所有帧 (原始图像列表)。

.image 最后一帧 (`.images` 的最后一个元素), 方便 “只看一张” 的场景。

.plot 把原始图像画出来的便捷方法。它**只画原始像素, 绝不叠加任何标注**——没有站点圆圈、没有阈值、没有检测结果。

9.1.3 核心不变量: capture 只显示原始图像

最重要的一条边界

`exp.capture()` (以及 `CaptureResult.plot`) **只显示原始图像**。站点圆圈 (site circles)、阈值 (thresholds) 这些都属于读出层 (sitmap / detect), **永远不属于相机**。如果你发现一次 `capture()` 居然画出了圆圈或阈值, 这**不是一个显示选项问题**, 而是一个架构 bug 的征兆: 说明标定状态 (calibration state) 被错误地放进了相机层。正确的做法是把这些标定/检测逻辑移回 readout 层。

为什么如此强调这条不变量? 因为相机层一旦 “知道” 站点在哪里、阈值是多少, 它就和某次特定标定耦合了, 换一组 trap、换一次对焦都会让相机层失真, 且会让 “原始数据” 不再纯净——你无法再相信 `exp.capture()` 给你的是探测器真实读到的东西。把相机层保持为 “只产出原始像素”, 才能让下游的 sitmap/detect 自由演化、可重复标定。

9.2 相机与时序器的协作: arm-before-fire

相机自己**不会**凭空拍照; 在真实实验里, 曝光由时序硬件用外触发 (external trigger) 精确控制。但**相机不驱动时序器**——它是纯取帧器: `arm` 之后就只等触发、按边沿逐帧读出。是**测量层**编排两者, 且整个系统只有一个编排入口: `operations.measurement.triggered_frames(camera, sequencer, sequence, frames)`。这条次序叫 **arm-before-fire**, 保证相机在第一个触发到来前已就绪、绝不漏帧。

一次带时序的采集, 流程如下:

1. **相机武装**: `camera.arm(frames)`——返回时硬件已进入外触发等待, 准备接 `frames` 个触发。
2. **测量层 fire**: `sequencer.prepare(seq)` 上传边沿表、`sequencer.fire(seq)` 启动序列。因为 fire 发生在 `arm` 返回之后, 它永远追不上相机——第一个边沿必然被接住。
3. **取帧**: `camera.read_frames(frames)` 从相机自有缓冲阻塞取帧。武装期间到达的帧**全部无损排队**, 即使序列先跑完、消费方后来取 (迟到消费) 也一帧不丢。
4. **收尾**: `camera.disarm()`; 需要时 `sequencer.wait_done()` 确保序列跑完。

注意职责划分: 相机被动等触发, **prepare/fire 只出现在 triggered_frames 一处、由测量层调**, 相机本身从不碰时序器。`frames` 是测量层想要的帧数 (它知道自己的脉冲带几个相机触发), 相机**不把**单触发序列改写成 `frames` 个触发。这样虚拟与真机走同一条契约路径, 换真机只换最底层的相机帧来源。

Python

```
from Zou_lab_control.neutral_atom.operations.measurement import
↳ triggered_frames
```

```
# 测量层编排: arm 相机 -> fire 序列器 -> 从相机缓冲取帧 (arm-before-fire)
raw = triggered_frames(camera, sequencer, imaging_seq, 3)
```

常见错误

不要先 `sequencer.fire()` 再 `camera.arm()`——那样相机还没武装、第一批触发就丢了。也不要再在调用方手拼 `prepare+fire` 包住相机：唯一合法的配对处就是 `triggered_frames`（契约测试会 `grep` 编排层抓违例）；相机层则永远只做取帧。

9.3 曝光即真值 (exposure-as-truth)

这是一个容易搞反的设计原则：**曝光时间的真值来自序列，而不是相机。**

具体地说，成像曝光由**探测脉冲的宽度** (probe pulse width) 决定——原子在多长的探测光脉冲里发荧光，相机就应该在多长时间里收集这些光子。因此正确的做法是：

真值在序列 曝光等于序列里探测脉冲的宽度。相机的 `exposure` 应当**从序列取值**，而不是你在相机上设一个数、反过来去影响序列。

修改曝光的正确入口 要改曝光,改序列:调用 `exp.timing.configure_imaging(exposure=...)`。这会更新成像序列里探测脉冲的宽度，相机随之得到正确的曝光窗口。

为什么不直接 `camera.configure(exposure=...)`?

你当然可以用 `configure(exposure=...)` 设相机自身的曝光数值（例如纯软件触发、无序列的快速调试）。但在真实外触发实验里，曝光窗口由触发脉冲宽度决定。如果你只改相机数字、不改序列，二者就会不一致：相机以为曝光是 A，硬件实际给的探测脉冲是 B，得到的荧光量与你的预期对不上。把序列当作单一真值源 (single source of truth)，用 `exp.timing.configure_imaging(exposure=...)` 统一修改，就不会出现这种漂移。

9.4 两种实现：qCMOS 真机 vs Virtual 仿真

同一个 `CameraDevice` 契约有两个实现。它们对上层完全可替换，但内部机制和适用场景不同。

9.4.1 QCMOSCamera (`qcmos.py`): 真实硬件

`QCMOSCamera` 封装了 Hamamatsu qCMOS 相机，走 DCAM SDK，使用**外触发**采集。它的配置由 `QCMOSConfig` 描述：

exposure 曝光时间，默认约 20 ms。

readout_speed 读出速度 (DCAM 的读出模式)。

roi 感兴趣区域 (Region Of Interest)，只读出传感器的一块子区域以提速/省数据。

device_index 设备索引，用于在多台相机时选定哪一台。

timeout_ms 等待触发/读出的超时 (毫秒)。外触发场景下，如果序列没发触发或发晚了，相机会在这个时间后超时报错，而不是无限等待。

它的生命周期是一个明确的状态机：

Python

```
open()      # 打开 DCAM 设备, _dcam 变为有效句柄
configure() # 下发 exposure / readout_speed / roi 等
acquire()   # 进入外触发等待并采集
close()     # 关闭设备, 释放句柄
```

_dcam 的语义 当相机处于关闭状态时, `_dcam is None`。这是判断“相机是否已打开”的内部标志：在 `open()` 之后它是一个有效的 DCAM 句柄, 在 `close()` 之后又回到 `None`。如果你在 `_dcam is None` 的状态下尝试采集, 会失败——必须先 `open()`。

控制流顺序不可乱

真机的标准顺序是 `open() → configure() → acquire() → close()`。在没 `open()` 之前 `configure/acquire` 没有有效句柄可用；忘了 `close()` 会占住设备让下次 `open()` 失败。把 `acquire` 放在 `open` 与 `close` 之间, 是使用真机的硬性要求。

9.4.2 VirtualCamera (virtual.py): 仿真相机

`VirtualCamera` 让**整条读出流水线在完全没有硬件的情况下也能跑**, 对开发、测试、CI 极其有用。它不接触任何真实设备, 而是**合成图像**：

- 通过 `trap_array.render_image()` 渲染合成图。
- 每一个被占据 (occupied) 的站点画成一个高斯点扩散函数 (Gaussian PSF) ——模拟一个原子的荧光像。
- 叠加泊松散粒噪声 (Poisson shot noise), 逐原子、逐帧采样, 使每帧都有真实相机那样的统计涨落。

这样产出的图像虽是合成的, 但格式与真机一致 (原始像素), 所以 `sitemap/detect` 等下游照常工作, 你可以在没有原子、没有相机的桌面上完整验证读出逻辑。

9.4.3 仿真模型与参数 (脉冲驱动, 对标真实 Rb87 v16 qCMOS)

仿真不是“随机画几个点”：**每一帧相机图就是一个实验循环**, 由实际 fire 的 `PulseSequence` 逐帧驱动。一条序列里每个相机触发 (如 `emCCD`) 边沿对应一帧, 相邻两个触发之间的脉冲被解析成该帧的冷却时长、`trap-off` 间隙与曝光时长 (`cooling_durations_per_frame / trap_off_durations_per_frame / exposures_per_frame`)。于是装载率、温度 (release-recapture)、读出保真度都从**同一套物理里**涌现, 只能通过渲染出的相机帧“测”回来——分析层从不读这些真值, 与真机完全一致。下面逐个物理效应给出实现与**当前参数值及其物理依据**：

装载 (loading) 每个光阱独立按伯努利装载, 概率 $p = \text{loading_probability} * (1 - \exp(-t_{\text{cool}} / \text{load_time_constant_s}))$: 冷却/MOT 时间越长越饱和。`loading\_probability=0.5` (光助碰撞单原子装载上限)、`load\_time\_constant\_s=0.5\,ms`、`mot\_load\_s=0.30\,s` (一发虚拟 shot 代表一个真实量级的 MOT/PGC 循环)。所以**编辑脉冲里的冷却时长会真实改变图像里的装载**。

PSF 与信号 每个被占据站点画成各向同性高斯, `atom_sigma_px=0.7\,px` (v16 逐站点 PSF 拟合 $\sigma_x \sim 0.61 / \sigma_y \sim 0.76$, 几何均值 ~ 0.68) ; 峰值幅度 = `atom_rate` × 散射时间,

`atom_rate=1100\,ph/s` (对标 v16 亮原子在 2 ms 参考读出下的探测光子率 $\sim 4.6/4.0$ kph/s, 使每原子光子数在同曝光下与 v16 一致而不过亮——过亮会让直方图过分分离、读出显得比真实 Rb87 容易); 站距 `spacing_px=9.0\,px` (v16 ~ 9.2)。

背景 杂散光 + 散射底噪 `background_rate=300\,ph/s/px` (对标 v16: 2 ms box 在 49 px (7×7) 上约 29 个探测光子, 即 ~ 0.6 光子/px/2ms $\rightarrow \sim 300$ ph/s/px), 故暗站点也有真实散粒底噪而非近零; 太高的底噪 (旧的 600) 会加背景噪声、封住可达保真度天花板; `dark_current_e_per_s=0.006`。

相机噪声 先对期望光子取泊松, 再 `counts = pe / conversion_e_per_count + offset_counts + N(0, read_noise_e / conversion)`。 `conversion_e_per_count=0.107`、`read_noise_e=0.43`、`offset_counts=200` —— 与真实 v16 CameraConfig 逐字一致。

读出损耗 / 寿命 每个原子在 probe 下寿命 $\sim \text{Exp}(\text{detection_lifetime} = 2.0\text{ s})$: 曝光越长散射光子越多 (SNR 越高) 但中途丢失越多——中途丢失的原子从**同一装载的下一帧**占据里掉出。这正是 detection-time 扫描测的 SNR-vs-存活权衡。

温度状态 每个原子带一个**逐站点温度**, 由脉冲逐帧演化: 装载/再冷却把它压向 PGC 底, probe 把它抬高, release-recapture 按它判存活。新装载起始于 `cooled_temperature_K=50\,\text{mu K}`。

冷却对温度的影响 一发**首帧**的冷却 = 新装载 (温度回到 PGC 底); **已装载原子**上再来一段冷却脉冲 (如 probe 加热后、release 前的 PGC) 则**再冷却**: $T \rightarrow \text{floor} + (T - \text{floor}) e^{-t_{\text{cool}}/\text{pgc_cool_tau_s}}$ (`pgc_cool_tau_s=0.3\,ms`), **不换原子**。所以“加热 $\rightarrow$ 再冷却 $\rightarrow$ 释放”能回到 PGC 底。

probe 加热对温度的影响 probe 以 `probe_heating_K_per_s` 反冲加热 ($T + = \text{rate} \times t_{\text{probe}}$)。默认 0 = **molasses 冷却成像** (成像光里带冷却, 反冲与冷却平衡, 读出对温度中性——这正是真机为何要 molasses 成像: 以真实反冲率不带冷却地成像几毫秒就会把 Rb87 加热到 $\sim$ mK 而抛掉原子)。调高即模拟**不冷却/molasses 不完美的**成像: probe 段抬高温度, 随后的 release-recapture 会读到更高温度 (不再冷却就直接丢原子)。

释放-再捕获 trap-off 持续 `t_off` 后原子按当前温度弹道飞散, 逐轴再捕获比例 $\text{erf}(r_c/(\sqrt{2}\sigma_v t_{\text{off}}))$ 、三维取立方, $\sigma_v = \sqrt{k_B T/m}$, `capture_radius_m=6\,\text{mu m}`、`recapture_mass_kg=Rb87` (1.443×10^{-25} kg)。温度越高、gap 越长, 存活越低——这就是温度测量的信号。

真空 trap 损耗 (暗保持) 两帧之间的**暗保持** (trap 开、probe 关) 按 $e^{-t_{\text{hold}}/\text{trap_lifetime_s}}$ 丢原子 (背景气体碰撞, 与光/温度无关), `trap_lifetime_s=30\,s`。几毫秒读出窗口对它几乎无损 ($\sim 0.01\%$), 但一个 hold-time 扫描就能测出 trap 寿命。

每帧效应复合 同一相机帧窗口内按时间顺序复合: 冷却脉冲 (首帧/重复序列 reload, 单循环多触发序列则**再冷却**) $\rightarrow$ trap-off 释放-再捕获损耗 $\rightarrow$ 暗保持真空损耗 $\rightarrow$ 成像 (散射 + 读出损耗 + 反冲加热) ——不会只取其一。

reload vs 再冷却的判定 单触发成像序列**重复**到 N 帧 (如阈值标定) = 每帧一发全新装载, 冷却即 reload; 而**单条多触发序列** (如 release-recapture bracket) 一发原子贯穿全部帧, 中途冷却即**再冷却**不 reload。按“基序列触发数是否已 $\geq$ 帧数”区分。

多帧 / 多 emCCD 触发 一条序列含多个相机触发即渲染多帧, 每帧各自按上面规则成像。

单一真相源

以上数值的**权威定义**在 `virtual.py` 的**字段注释** (本节是其镜像, 改值以代码为准, 不要在别处复制这些字面量)。要改仿真行为: 直接改 `virtual.py` 的字段, 或在连接时覆盖 `na.connect("virtual", sitemap=\{\dots\})` / 设备配置。

9.4.4 force_all_sites: 渲染满阵列 (仿真专属)

`VirtualCamera` 有一个仿真专属的设备属性 `force_all_sites` (默认 `False`)，用来显式控制“是否把所有站点都视为已装载 (loaded)”：

作用 需要一张“满阵列”图像做确定性的设备级测试时，设 `cam.force_all_sites = True`，渲染器就把每个站点都画成一个原子；平时保持 `False`，按本帧实际装载情况渲染。

生产路径从不使用它 `sitemap` 标定靠对多帧真实 ~50% 装载求平均找出全部站点——与真机走同一条路，不依赖任何仿真捷径。

与真机的本质区别 真实硬件没有这种捷径：要得到“所有站点都亮”的图像，必须真的把原子装载到每个站点上。仿真可以“假装”全装载，真机不行——所以别把这个开关写进期望在真机上同样生效的代码路径。

9.5 一个可运行的采集示例

下面给出一个从设定曝光到拿到原始帧的最小例子，串起本章所有概念。注意：先用序列设定曝光真值，再采集；带时序器的一发经 `triggered_frames` 编排；最后用 `exp.capture` 的 `.plot` 只看原始图。

Python

```
# 1) 曝光即真值：改曝光要改序列，而不是只改相机
# 这会更新成像序列里探测脉冲的宽度 (= 曝光窗口)
exp.timing.configure_imaging(exposure=20e-3) # 20 ms

# 2) 高层采集：会话编排一发并打包成 CaptureResult
result = exp.capture(frames=3, display=True)

# CaptureResult 的常用字段：
all_frames = result.images # list[np.ndarray], 全部原始帧 (uint16)
last_frame = result.image  # 最后一帧 = images[-1]
result.plot()              # 只画原始像素，绝不叠加圆圈/阈值

# 3) 若需要底层、解耦式的采集 (readout 流水线内部用法)：
# 相机是纯取帧器，绝不碰时序器；测量层唯一的编排 helper 完成
# arm 相机 -> fire 序列器 -> 从相机缓冲取帧 (arm-before-fire):
from Zou_lab_control.neutral_atom.operations.measurement import
↳ triggered_frames
raw = triggered_frames(camera, sequencer, imaging_seq, 3)
# raw 是 list[np.ndarray], 每个元素是一帧 uint16 原始像素
```

自检清单

- `exp.capture()` 画出来有圆圈或阈值？——架构 bug，标定状态被放进了相机层，应移回 `readout (sitemap/detect)`。
- 想改曝光却去调 `camera.configure(exposure=...)`？——在外触发实验里请改序列：`exp.timing.configure_imaging(exposure=...)`。
- 真机采集报句柄无效？——检查是否漏了 `open()` (此时 `_dcam is None`)，顺序应为 `open→configure→arm/read→close`。
- 仿真想要满阵列图？——设备级测试用 `cam.force_all_sites = True`；真机则必须真正装载原子。

- 相机没出帧 / 帧数对不上？——相机是纯取帧器，绝不自己驱动时序器：外触发一发由测量层的 `triggered_frames` 编排 (`arm` → `fire` → `read`)。别在调用方手拼 `prepare/fire`，也别指望相机把单触发序列改写成 `frames` 个触发——它就按外触发逐帧取。

9.6 第二只相机：MOT 监视相机与线圈磁场寻优

除了负责读出的主相机 (“`camera`”), 设备清单里还可以有一只**监视相机** (约定名 “`monitor_camera`”): 它不参与站点读出, 只盯着 MOT 的荧光斑。典型用途是**扫 $x/y/z$ 三路线圈电流 (DAC 码), 同时看 MOT 亮度, 找出装载最好的磁场**。监视相机与主相机遵守**完全相同**的纯取帧器契约 (`arm` / `read\_frames` / `disarm`, 经 `triggered_frames` 完成 `arm-before-fire`), 所以扫描引擎、面板、处理器对“哪只相机”完全无感。

9.6.1 两种实现：PylonCamera 真机 vs VirtualMotCamera 仿真

PylonCamera (`devices/pylon.py`) Basler pylon 相机的薄封装 (`pypylon` 在 `open()` 内惰性导入, 没装 Basler 运行库的机器照样可以导入包、跑虚拟后端)。`trigger\_source` 选外触发线 (如 “`Line1`”) 或 “`Software`” 自由运行; `serial` 在多相机时选定一台。真机拍的是真实世界——脉冲到达它的唯一途径就是物理触发线。

VirtualMotCamera (`devices/virtual.py`) 虚拟 MOT 物理相机。它**只感知真正 fire 出去的序列**: 在帧中点时刻用 `decode\_analog\_bus` (`timing/sequence.py`, DAC 编码链的精确逆变换) 从编译后的位通道脉冲里重建每路线圈总线的带符号电平, 再按三维高斯模型算 MOT 俘获效率 $\eta = \exp\left(-\frac{1}{2} \sum_i \left(\frac{L_i - B_{0,i}}{\sigma_i}\right)^2\right)$, 渲染一颗中心高斯荧光斑 (泊松光子噪声 + 读出噪声 + 常数 `offset`, 与 qCMOS 噪声链同构)。**它从不读任何人的设定值**——虚拟与真机走同一条“编译产物”路径, 换真机只改 `connect()` 的配置。

9.6.2 自动发现与一键接线: `discover\_devices`

接真机的第一步不是抄序列号, 而是**扫一遍总线**:

Python

```
import Zou_lab_control.neutral_atom as na

found = na.discover_devices()          # 打印一张表并返回行对象
# [basler] 24012345   acA1920-155um (ready)
# [ visa] GPIB0::17::INSTR Rigol,DSG836,... (-)

cams = [d for d in found if d.config is not None]
devices = na.load_devices({"monitor_camera": cams[0].config})  #
↪ 片段直接可用
```

规则与 confocal 参考实现一致: 枚举 Basler (`pypylon`) 相机与 VISA 资源 (逐个 `*IDN?` 短超时问身份); **缺库或空总线是一行提示, 绝不抛异常**。每台 Basler 相机的行自带 **ready config 片段** (一台钉死 `serial`、`Software` 自由跑的 `PylonCamera`), `load\_devices` 原样消费——发现结果与设备构造之间没有手工转抄。裸 VISA 仪器本包没有对应设备类, 只列地址 + 身份供你接进自定义类 (`register\_device\_class`)。

再快一步: 自带设备图 “`basler\_monitor`” **只声明真实存在的硬件**——一台自由跑的真 Basler (`serial=""` 取第一台), 别无他物。缺的角色 (读出相机 / `sequencer` / `trap`) 就是缺: `session` 对缺角

色宽容，存在的角色自然点亮，config 里**绝不用**假设备凑数（这正是设备层的解耦契约——声明什么就是什么）。插上 USB：

Python

```
exp = na.connect("basler_monitor", open_devices=True)
exp.capture(camera="monitor_camera", frames=1, display=True) # notebook
↪ 直接看图
```

控制台同一路径：task_console.bat -config basler_monitor 启动，Add Panel → Measurement → **Camera (live frames)**，Camera 下拉切 monitor_camera，Start 即实时看图（该下拉与 Pulse scan 的相机选择同出一源 DeviceSet.camera_names()）。自由跑模式（Software 触发）不需要任何脉冲接线；接好 FPGA 触发线后把 trigger_source 改 "Line1"，其余代码全部不动。acA1920-155um 的 pixel_format 可选 "Mono8"（快）或 "Mono12"（动态范围）。

9.6.3 脉冲模板：三个 DAC api 槽就是三路线圈

线圈扫描用普通的脉冲模板表达：三条 6 bit 模拟总线（如 da_x/da_y/da_z），每条在成像 period 上绑一个 **dac api 槽** (a1/a2/a3)。仓库自带 pulses/mot_field_template.json：period0 让线圈稳定（1 ms），period1 曝光（5 ms，mot_trigger 高电平触发监视相机）。自己搭模板时在 pulse GUI 里把 DAC 单元格点到 API（紫色）态即可。

9.6.4 MOT 光强：一条单源规则

MOT 亮度由 mot_roi_intensity(frame, cx, cy, radius)(operations/processors/mot_intensity.py) 定义：**圆盘 ROI 内均值减去外环 (r..2r) 背景均值**，环境光与相机 offset 自然抵消。控制台的 **MOT intensity** 处理器（Add Panel → Processor）与一键寻优 task 消费的是**同一个函数**——实时监看与优化器不可能意见不合。处理器接受一张 (H, W) 帧或相机的 (repeat, 1, H, W) 块，输出每次采集一个标量 mot_intensity (repeat_contract="reduce")。

9.6.5 手动扫：Pulse scan + monitor_camera + grid facet

想边扫边看，用通用的 **Pulse scan** 测量即可：camera 参数选 monitor_camera，扫描模式选 API（软件扫，逐点 set_api 重编译再 fire），y 选 MOT intensity 处理器发布的 mot_intensity。三维网格在 api 扫描程序里声明 scan_shape，与硬件扫描表**同一份声明**，grid 面板据此把三维块 facet 成一格一平面：

Python

```
import numpy as np
a = np.array([1, 4, 7, 10, 13]) # Bx 候选码 (带符号 LSB, 0 = 0 V)
b = np.array([-11, -8, -5, -2, 1]) # By
c = np.array([5, 8, 11, 14, 17]) # Bz
A, B, C = np.meshgrid(a, b, c, indexing='ij')
scan_table = np.column_stack([A.ravel(), B.ravel(), C.ravel()])
scan_shape = (len(a), len(b), len(c)) # 声明网格 -> grid 面板 facet
↪ 成一格一平面
```

9.6.6 一键寻优：Optimize MOT field task

`OptimizeMotFieldTask`(`operations/tasks/mot\_field.py`, 控制台里叫 **Optimize MOT field**) 把整条链打包成一次运行：按 `center\_x/y/z` 与 `span、points` 生成三维码网格，逐点 resolve 三个 dac api 槽、fire、在监视相机上量 MOT 光强；寻优 = argmax 加 3^3 邻域质心细化（免拟合、抗散粒噪声，可到亚步精度）；报告落到 `folder`：原始强度块 `mot\_field\_scan.npz` + 一张按 B_z 平面 facet 的图。`result` 给出 `best\_x/best\_y/best\_z`（细化后的最优码，可以不落在网格点上）与 `best\_intensity`。运行中 `frame/progress/stage` 走 task 输出通道，Monitor 面板实时可见。

自檢清單

- task 报“must expose exactly THREE dac api slots”? ——模板没把三路线圈绑成 dac api 槽；到 pulse GUI 把三个 DAC 单元格点到 API（紫色）态再存模板。
- 扫描值被拒“value must be between ...”? ——DAC 码超出总线位宽的带符号范围（6 bit 即 $-32..31$ ）；网格取值要在范围内。
- 虚拟端 MOT 亮度不随扫描变化? ——检查扫的是不是模板里真实绑定的那三个槽；`VirtualMotCamera` 只认 fire 出去的序列里解码到的电平（`snapshot()` 的 `last\_levels` 可以直接看它感知到了什么）。
- 手动扫时 y 一直空? ——MOT intensity 处理器的 Frame source 要选**扫描节点发布的** `frame\_0`（Pulse scan 自己发布帧），不是别的相机测量的帧。

10

相机读出教程与原理（深入）

本章从零讲解中性原子实验里的**相机读出** (camera readout)：如何把相机拍到的一帧二维荧光图像，转换成“哪几个光阱里有原子”这一组布尔判定。整条链路都挂在会话对象 `exp.readout` 上，它是一个 `ReadoutSubsystem` 实例(定义在 `Zou\_lab\_control/neutral\_atom/subsystems/readout.py`)。底层的纯数学函数集中在 `Zou\_lab\_control/neutral\_atom/core/analysis.py`，标定结果的容器是 `Zou\_lab\_control/neutral\_atom/core/calibration.py` 里的 `TrapCalibration`。

读懂本章后，你应当能独立完成一次完整读出：先做三步**标定** (site-map → thresholds → 可选的 `detection_time`)，再做**逐帧探测** (detect)，并能解释每一步背后的物理与统计原理、每个参数的含义、返回值的结构，以及最常见的几类失败如何排查。

10.1 总体框架：标定一次，探测无数次

读出的核心思想是把“昂贵、需要假设”的工作和“廉价、每发都做”的工作分开：

标定 (calibration) 偶尔做一次，回答两个问题：每个光阱在相机像素坐标里的中心在哪里 (site-map)；以及每个光阱“暗”和“亮”之间的计数分界线在哪里 (thresholds)。标定通常需要一个**特殊的假设** (比如“所有光阱都装满原子”)，所以它**不是测量，而是定标**。

探测 (detection) 每一发 (shot) 都做：拿到一帧图像，在每个已知中心周围数光子计数，把计数和阈值逐位比较，得到一个布尔占据向量。它是一个**纯阈值二分类**，每一发完全独立，不依赖前后历史。

三步标定，每一步都产生一个 `TrapCalibration` 并**原地存回会话** (store on the session)。后续的 `exp.readout.current` 永远指向“当前生效”的那份标定。三步是层层递进的：

1. `sitemap(...)` 找出 N 个中心坐标 `centers` (形状 $(N, 2)$ ，像素坐标)。
2. `thresholds(...)` 在已知中心的基础上，给每个位点定一条计数阈值 `thresholds` (形状 $(N,)$)。
3. `detection_time(...)` (可选) 扫描成像曝光时间，找到一个保真度 (fidelity) 饱和的好成像时间。

只有 site-map 之后才能做 `thresholds` (thresholds 需要中心坐标才能数 ROI)；只有 thresholds 之后

才能做 detect(detect 需要阈值才能分类)。这条依赖链在代码里由 `require(thresholds=True/False)` 守护。

一句话记住流程

`sitemap` 回答“原子可能在哪”，`thresholds` 回答“多亮才算有原子”，`detect` 回答“这一发到底哪几个有原子”。

10.2 第一步: site-map 标定 (找中心)

10.2.1 调用与返回

Python

```
res = exp.readout.sitemap(
    frames=20,          # 平均多少帧来压噪声
    grid_shape=(5, 7),  # 光阱排成 5 行 7 列, 共 35 个位点
    ordering="row-major", # 位点编号顺序
    roi_radius=1,        # 每个中心周围 (2*1+1)=3x3 的方框
    reducer="mean",      # 把方框里的像素压成一个标量的方式
    display=True,        # 画出带圆环的标注图
)
```

返回一个 `SitemapResult`, 关键字段:

res.calibration 一个 `TrapCalibration`, 其中 `res.calibration.centers` 是形状 `(N, 2)` 的像素坐标数组, 每行是一个 `(x, y)`。这份标定已经被原地存回会话, 等价于 `exp.readout.current`。

res.average_image 把 `frames` 帧逐像素平均后的二维图 (信噪比比单帧高)。

res.images 用于平均的原始帧堆栈, 方便事后复查。

res.plot 当 `display=True` 时给出的标注图: 在平均图上, 每个找到的中心套一个圆环。这是你**第一眼必须看的东西**——肉眼确认圆环都落在真实亮斑上、数目正确、顺序合理。

10.2.2 参数逐个解释

frames 平均的帧数。平均是为了压低读出噪声与散粒噪声: N 帧平均后噪声标准差约降到 $1/\sqrt{N}$ 。20 帧是一个稳妥起点; 亮斑很弱时可以加大。

grid_shape `(ny, nx)`, 即行数乘列数。它决定要找的位点总数 $N = ny \times nx$, 也决定排序逻辑怎么把散乱的点切成网格。**这个数必须和真实光阱排布一致**, 否则要么找不够、要么排错。

ordering 位点的全局编号顺序, 见下面“位点排序”小节。它决定 `centers` 里第 0、1、2... 行对应物理上哪个阱, 进而决定后续所有 `counts`、`occupied` 向量的下标含义。

roi_radius 即底层 `roi_counts` 的 `radius`。中心 `(cx, cy)` 周围取一个 `(2*radius+1) x (2*radius+1)` 的正方形方框 (`radius=1` 即 `3x3`)。半径太小会漏掉原子荧光的尾巴, 太大会把邻阱的光也数进来。

reducer 把 ROI 方框里那一小块像素压成一个标量的方式, 支持 `"mean"`、`"sum"`、`"median"`、`"max"`。
`mean` 最常用、抗噪; `sum` 与曝光线性成比例; `median` 抗坏点; `max` 对热点敏感。**这个选择必须在 `thresholds` 和 `detect` 之间保持一致** (见后文一致性约束)。

`display` 是否生成 `res.plot` 标注图。批量脚本里可设 `False` 省时。

10.2.3 原理: `find_site_centers` 怎么找中心

底层是 `core/analysis.py` 里的 `find_site_centers`。直觉上它在做“在一张全亮的图里挑出 N 个最显眼的山峰, 再把山峰排进网格”。具体步骤:

1. **平均**: 把 `frames` 帧平均成一张干净图 (信噪比更高)。
2. **高斯平滑**: 用 `gaussian_filter(sigma=1.0)` 轻度模糊。把单像素噪声尖峰抹平, 让每个原子荧光斑变成一个圆润的单峰, 避免一个阱被误判成好几个峰。
3. **找局部极大**: 用 `maximum_filter` 找局部极大值点 (一个像素等于它邻域里的最大值, 就是候选峰)。`min_distance` 控制邻域大小, 默认按图像尺寸和网格密度自动估算 (约为相邻阱间距的一半), 确保不会把同一个斑找成两个。
4. **相对阈值过滤**: 计算 `cutoff = min + threshold_rel*(max - min)`, 默认 `threshold_rel=0.35`, 即只保留亮度高于“动态范围 35%”的候选峰。这把背景里的伪峰滤掉。
5. **兜底**: 如果通过阈值的候选少于 N , 就退化为“直接取整张平滑图里最亮的 N 个像素”, 保证总能凑够 N 个点 (但这通常意味着信噪比不够好, 应当警觉)。
6. **按亮度取前 N** : 候选按峰值亮度排序, 取最亮的 N 个。
7. **排序进网格**: 交给 `sort_centers_grid` 按 `ordering` 排成稳定的位点顺序。

最关键的假设

site-map 假设所有光阱都被占据。它本质是在一张“满阵列”图上定位, 所以它是**定标而非测量**——你不能用 site-map 的结果去回答“现在有几个原子”, 那是 `detect` 的工作。做 site-map 时务必让阵列尽量全满、亮度均匀。

10.2.4 位点排序: `row-major` / `serpentine` / `column-major`

`ordering` 决定 `centers` 的行顺序, 也就决定了你后面看到的“第 0 号阱、第 1 号阱...”在物理上是谁。在 `analysis.py` 中支持的取值 (含别名):

“row-major” 先按 y 切成 n_y 行, 每行内部按 x 从左到右。编号像读书: 左上 $\rightarrow$ 右, 再下一行左 $\rightarrow$ 右。最常用、最好理解。

“serpentine” 蛇形 (snake): 奇数行反向。即第一行左 $\rightarrow$ 右, 第二行右 $\rightarrow$ 左, 第三行左 $\rightarrow$ 右.....适合物理上按蛇形寻址的系统。

“column-major” 先按 x 切成 n_x 列, 每列内部按 y 。编号按列走。

排序的实现思路: 先按一个轴排序再 `array_split` 切成等份 (行或列), 份内再按另一个轴排序, 蛇形时把奇数份反转。**选定一种 `ordering` 后就别再改**, 否则同一个 `exp` 上前后做的标定下标含义会错位。

10.3 第二步: thresholds 标定 (定亮暗分界)

10.3.1 调用与返回

Python

```
res = exp.readout.thresholds(
    frames=100,      # 采多少帧来建直方图 (越多分界越稳)
    site=0,          # display 时重点画哪个位点的直方图
    exposure=None,   # 成像曝光时间; None 用会话默认
    display=True,
)
print(res.thresholds)      # 形状 (N,), 每个位点一条阈值
print(res.selected_site)   # = site, 被重点展示的位点
print(res.fidelity)       # 该位点暗/亮分布的保真度估计
```

返回 `ThresholdResult`, 关键字段:

res.calibration 更新后的 `TrapCalibration`, 现在它**同时有** `centers` 和 `thresholds`。这份已存回会话, `exp.readout.current` 指向它。

res.thresholds 形状 `(N,)`, 每个位点一条标量阈值。与 `res.calibration.thresholds` 同值。

res.counts 形状 `(shots, sites)`, 即 `frames` 帧 $\times$ `N` 位点的逐帧逐位计数原始数据, 可用来自己重画直方图。

res.selected_site 等于入参 `site`, 仅用于 `display` 聚焦展示某个位点。

res.fidelity 选中位点的读出保真度估计 (见原理小节)。

10.3.2 原理: 从直方图到 Otsu 阈值

对每一帧、每一个位点, 先用 `roi_counts` 在该中心周围取 `(2*radius+1)` 见方的方框, 按 `reducer` 压成一个**标量计数**。把 `frames` 帧累起来, 每个位点就有一串计数样本。这串样本天然**是双峰**的:

暗峰 (dark) 阱里**没有**原子时, 方框里只有背景光读出噪声, 典型计数约 `0-5`。

亮峰 (bright) 阱里**有**原子时, 原子荧光叠在背景上, 典型计数约 `50-200`。

把每个位点的计数装进 `96` 个直方图 bin, 再用 `otsu_threshold` 求分界。Otsu 法的数学直觉: 在所有可能的分界线里, 挑那条让**类间方差最大** (等价于类内方差最小) 的。设分界把样本分成“暗类”和“亮类”, 权重各为 ω_0, ω_1 ($\omega_0 + \omega_1 = 1$), 均值各为 μ_0, μ_1 , 类间方差正比于 $\omega_0 \omega_1 (\mu_1 - \mu_0)^2$ 。Otsu 扫遍所有 bin 边界, 取让这个量最大的那条。直觉上它会落在双峰之间最深的“山谷”里。典型阈值约 `20-50`, 正好卡在暗峰和亮峰中间。

代码里 `otsu_threshold` 还做了健壮性处理: 先滤掉非有限值; 若所有值都相同则直接返回该值; 用累积概率 ω 与累积均值 μ 向量化计算每个候选分界的类间方差分数, 取 `argmax` 对应的 bin 中心。

为什么是 96 个 bin

bin 太少会让阈值“量化”得太粗 (分辨不出谷底); bin 太多则每个 bin 样本太少、直方图噪声大、谷底飘。96 是在“分辨率”和“统计平滑”之间的折中。`frames=100` 提供大约 $100 \times N$ 个样本来填这些 bin。

10.3.3 保真度：estimate_threshold_fidelity

`res.fidelity` 来自 `estimate_threshold_fidelity(values, threshold)`。原理是把一条计数分布按给定阈值劈成左（暗）右（亮）两半，对每半各拟合一个高斯（取均值 μ 与样本标准差 σ ），然后算两个错判概率：

- 暗被误判成亮的概率 = 暗高斯落在阈值右边的尾巴；
- 亮被误判成暗的概率 = 亮高斯落在阈值左边的尾巴。

两个高斯尾巴的重叠越小，保真度越高。代码用正态分布 CDF (`_normal_cdf`，基于误差函数 `erf`) 算这两个正确判定概率，按两类样本数加权得到原始保真度 `raw_fidelity`。此外还乘了一个置信度修正：用分离度 `separation = |mu1-mu0| / sqrt(s0^2+s1^2)` 和左右两类的平衡度，把“样本太偏、两峰太近”时虚高的保真度往 0.5 拉回，避免给出过于乐观的数字。返回的 `FidelityEstimate` 里还带 `dark_mean/bright_mean/dark_sigma/bright_sigma/separation` 等诊断量，便于你判断“是真分得开，还是只是样本太少凑出来的”。

一个好的读出，单位点保真度通常应在 0.99 以上；如果 `res.fidelity` 偏低，往往是成像时间不够（亮峰不够亮）或背景太高（暗峰漂移）。

10.4 进阶读出：PSF 加权提取 + 双高斯阈值 (Rb87)

方框计数 + Otsu 是默认方法，简单稳健。但在低光子数（成像时间短）或弱信号下，方框把信号像素和背景像素等权相加，信噪比会被边缘背景拖低（位点密集、PSF 边缘交叠时尤甚）。Rb87 qCMOS 读出用两点改进，二者都接在同一套 `TrapCalibration.signals/detect` 契约后面，方框法仍是默认，可逐项开启；`detection_time` 时间扫描也会跟着用标定自己的提取方法。

10.4.1 PSF 加权提取：method="psf"

Python

```
sm = exp.readout.sitemap(method="psf", psf_half_width=3, frames=20)
```

`sitemap(method="psf")` 在找出中心之后，额外从全亮模板（所有位点都有原子时的多帧平均）里给每个位点拟合一张归一化的 PSF 权重图（ $(2 \times \text{psf_half_width} + 1)$ 见方，默认 7×7 ）。逐发提取时，信号 = 该位点 box 内背景扣除后的像素与这张权重图的点积（匹配滤波）。匹配滤波是“已知形状、加性噪声”下信噪比最优的线性提取：靠近原子的像素权重重大，远离的接近 0，背景被自然压制。逐发提取默认用环形本底（box 外圈像素的中值）做局部背景扣除，对慢漂移背景自适应。标定对象因此多带 `psf_weights (N,h,w)` 与 `psf_boxes (N,4)` 两个字段，`save/load` (JSON 与 npz) 都会原样往返。无 session 时直接用 standalone 接口：`na.calibrate_sitemap_from_images(images, grid_shape=..., method="psf")`。

贴边站点会报错

PSF 读出要求每个站点都能取到完整的 $(2 \times \text{psf_half_width} + 1)$ 见方 box；若某站点离图像边缘太近、box 被裁，`fit_site_psfs` 会直接 `raise`（而不是悄悄用一个小 box）。必要时减小 `psf_half_width`，或确保成像视场对每个阱位都留足边距。

10.4.2 双高斯阈值：method="bimodal"

Python

```
th = exp.readout.thresholds(method="bimodal", frames=120)
```

`thresholds(method="bimodal")` 不再只取一条 Otsu 谷底，而是：先用精确 1D Otsu 在排序样本上做一次经验劈分，再对暗、亮两侧各自估计单侧峰核的均值与宽度（只用远离峰间桥的那一侧分位数，避免部分丢失/弱发的“桥”样本把亮峰宽度撑大），最后把阈值放在两个高斯总错判率最小的点上，并按模型给出 $0.5 \times (F_{\text{dark}} + F_{\text{bright}})$ 的保真度。阈值仍是逐位点的 $(N,)$ 数组；提取用的是该标定自己的方法 (`calibration.signals`)，所以 PSF 标定下阈值天然定在 PSF 信号上，和 `detect` 完全一致。某位点样本退化拟合不出时，会自动回退到该位点的 Otsu 阈值，保证标定全有限。

取原则，不照搬

这套实现移植自 `references/.../rb87\_readout\_v16` 的算法 (PSF 拟合、双高斯定阈、匹配滤波提取)，但参考归档里那套面向离线批处理的文件 IO / 流式缓存 / 输出目录命名 / 重复别名没有带进来——在线控制库只要算法，按本仓库的分层（原语在 `core/`，标定在 `operations/`，编排在 `ReadoutSubsystem`) 重写。

10.4.3 用实验数据逐站点定阈值 + 保真度：characterize_from_dir

前两步给的是“一条分布定一条阈值”。Rb87 真正的做法是用参考帧给出的 `ground-truth` 标签来逐站点训练并诚实评估阈值。这一步走文件夹：实验把相机原始帧逐张存进一个数据文件夹、命名 `PREFIX<n>`；`exp.readout.characterize_from_dir(data_dir, prefix)` 指向那个文件夹跑完整流程，和真机一字不差（唯一差别是帧由真相机还是 `na.write_virtual_run` 写的）。

Python

```
report = exp.readout.characterize_from_dir(
    data_dir, prefix="img",      # 文件夹里已有 img0, img1, ... 原始帧
    train_fraction=0.9, seed=1, # 随机划分逐站点训练 / held-out 评估
    # 下列默认对齐 Rb87 v16, 按需覆盖:
    # shots_per_group=4, short_shot=3, ref_shots=(1, 2, 4),
    # max_groups=None, store_thresholds=True, results_dir=None, save=True,
)
report.summary()                # aggregate/global/min 保真度、ambiguous 比例
```

`index_run` 把文件夹里的 `PREFIX<n>` 帧按 `shots_per_group` 一组索引（同一批原子的一次装载）：`short_shot` 是待表征的短读出帧，`ref_shots` 是高 SNR 参考帧（投票出真值标签）。流程：参考帧逐站点双高斯拟合 → 严格共识（一组的所有参考帧同亮/同暗才算定，否则 ambiguous）给出 `(group, site)` 真值标签 → 在随机 `train` 划分上为每个站点单独训练阈值 → 在 held-out `test` 上算保真度 → 另给一个全局单阈值对比 + drop-worst-site 消融。提取走当前标定的方法 (`calibration.signals`, `box` 或 `PSF`)，所以阈值定在 `detect` 实际用的信号上；因此先跑 `exp.readout.sitemap_from_dir(data_dir, prefix, method=...)` 从同一文件夹建好站点图 / PSF。

`store_thresholds=True` (默认) 把训练出的逐站点阈值写回标定 (`metadata["threshold_method"]="per_site_ref"`)，`save=True` (默认) 把信号、阈值、逐站保真度与 `summary()` 元数据写进 `results_dir` (缺省 `<data_dir>_results`)。

返回 `FidelityReport:per_site` (逐站 `SiteFidelity`)、`thresholds (N,)`、`site_fidelities (N,)`、`aggregate_fidelity` (逐站阈值 held-out)、`global_fidelity` (单阈值 held-out)、

`ablation` (drop-worst-k 行)、`labels/split`, 以及 `per_site_arrays()` 给绘图用的逐站 (`values`, `occupied`)。

10.4.4 逐站点网格: `grid(sub_plot_kind=...)`

把每个站点画成一张小图、按 trap 网格排布, 一眼看出哪个站点分得开、阈值卡得对。所有逐站点网格都走同一个 `zf.grid(...)` 工厂, 用 `sub_plot_kind` 声明每格是哪种 plot kind——“hist” 每格一张读出分布直方图、“2d” 每格一张图像 (如 PSF 核)。这一个声明同时驱动: 格内缩略图、双击放大成该 kind 的标准单图、以及 `figure_viewer` 里 Setting/Edit 显示该 kind 的参数 (hist 给 bins/fit/log 轴, 2d 给 colormap):

Python

```
values, occupied = report.per_site_arrays()
zf.grid(
    values, sub_plot_kind="hist",          # 每格 = 一张读出分布直方图
    occupied=occupied, thresholds=report.thresholds,
    site_fidelities=report.site_fidelities,
    grid_shape=exp.devices.trap_array.grid_shape,  # 例如 (5, 7) = 35 站
    labels=("PSF signal (counts)", "Shots"),
)

# PSF 核网格 (每格一张图像) = 同一入口, 只换 sub_plot_kind:
zf.grid(psf_weights, sub_plot_kind="2d", labels=("x (px)", "y (px)"))
```

`zf.site_histogram_grid(...)` 与 `zf.site_psf_grid(...)` 只是 `sub_plot_kind="hist"` / “2d” 的薄壳预设 (同一条真实路径)。hist 每格: 暗 = 灰、亮 = 蓝、橙线 = 该站阈值; 站号 (+ 保真度) 画在每格的小标题里、不占图内区域; 所有格共享 x 轴便于比较。这是 frontend 的通用多面板图 (站点数任意, 不写死 35), 建在 `canvas.create_axes_grid` 上——固定格 + 显式间距 + 图尺寸自适应, 因此不重叠、不裁切、对齐由构造保证 (见 `frontend/AGENTS.md` 的 Layout 节)。保存后可用 `exp.figure_viewer()` 重开: 双击任一格放大成该 kind 的标准图、再双击/Esc 返回, Flow 页始终画出数据从何而来 (至少 raw data → plot)。

facet: 把任意一根轴展开成网格

`grid` 同时是一个维度展开工具: 把测量块 (`repeat`, `points`, `*data\_dim`) 的某一根轴展开成 N 个格子, 每格是其余轴的一张标准小图。给 `zf.grid` 传整块数组 + `facet` (切哪根轴) 即可; `sub_plot_kind` 缺省时按每格剩余的形状自动选 (剩 2 维扫描网格 → “2d” 图像; 剩多点 → “1d” 曲线; 只剩重复采样 → “hist” 分布):

Python

```
# 3 层嵌套扫描 (scan_shape=(n0, n1, n2)) 的原始块 (repeat, n0*n1*n2, 1):
# 沿最外层扫描轴展开 -> n0 个格子, 每格一张 (n1, n2) 的 2D 扫描图
zf.grid(block, facet="points:0", points_shape=(n0, n1, n2),
    ↪ sub_plot_kind="2d")

zf.grid(block, facet="repeat")  # 每个 repeat 一格 (其余轴自动成形)
zf.grid(vec_block, facet="dim:0")  # 逐站点向量按站展开, 每站一张分布
```

在 task 控制台里同样直观: Add Panel 选 **Site grid** (新面板默认 `facet=repeat`, 绑上块信号立

即出图)，在 Setting 的 **facet** 下拉里选展开轴（选项由生产节点声明的轴结构自动生成，标注每根轴的长度；超过 80 格的轴会置灰——一次画太多格会卡死前端，请换轴或用表达式先折叠；**(saved figure)** 选项只在绑定的是载入保存图的节点时出现），**sub plot** 下拉选每格画什么（**auto** 按剩余形状自动，或显式 hist/2d/1d）——Setting 的参数行随解析出的每格类型自动切换（hist 网格给 bins/fit/ylog，2d 网格给 colormap）。hist 网格的 fit 与单面板 dis 是**同一份拟合**（同色三条曲线画在每个缩略图上），因 N 格逐格拟合有开销而默认关闭，需要时在 Setting 打开即可。相机块 (**repeat, 1, H, W**) facet 到 repeat 就是**每个 repeat 一张帧图**；面板的 Edit 页照常给出当前数据的交互快照，参数行同样跟随每格类型。此后每个 shot 网格**就地增量刷新**（artist 复用，绝不重建）；双击放大后 live 更新只喂放大图，Esc 返回时缩略图一次补齐——relim/fixed/cmap/bins/repeat_mode 等参数与所有 plot 同一机制。扫描程序里 **scan_shape** 可声明**任意层数**（如 **(5, 4, 3)**），流送的表保持扁平，只有显示层按声明还原。

10.5 第三步：detect 探测（逐发判定）

10.5.1 调用与返回

Python

```
res = exp.readout.detect(
    exposure=None,          # 成像曝光; None 用会话默认
    display=True,          # 在图上把"占据"的阱套亮环
    what="occupancy",      # 探测语义: 占据/不占据
)
print(res.detection.counts)          # (N,) 本发每位点计数
print(res.detection.occupied)        # (N,) bool 占据向量
print(res.detection.occupied_indices) # 占据位点的下标列表
```

返回 **DetectionResult**:

res.image 这一发拍到的原始二维图像。

res.detection 一个 **AtomDetection**，含 **counts**（每位点计数）、**occupied**（形状 **(N,)** 的布尔向量）、**occupied_indices**（占据下标列表）、**thresholds**（这次用的阈值）。

res.calibration 本次探测用到的 **TrapCalibration**（提供 centers / thresholds / reducer / radius）。

res.plot 当 **display=True** 时，在图上给**占据**的阱套亮环——一眼看出这一发装了几个、装在哪。

10.5.2 原理：TrapCalibration.detect 是纯阈值二分类

底层是 **TrapCalibration.detect(image)**（box 用法用标定里存的 centers / thresholds / radius / reducer）：

1. 用 **roi_counts** 在每个中心算出 **counts**（形状 **(N,)**），用法和标定时完全一样。
2. 把 **thresholds** 广播成长度 N（标量阈值会被扩成每位点同值）。
3. 逐位比较 **occupied = counts > thresholds**——**严格大于**才算占据。
4. **occupied_indices = flatnonzero(occupied)**，即占据位点的下标。

这是一个**逐元素的阈值二分类**，没有任何跨位点或跨发的耦合，**每一发独立**。它的判定质量完全由两件事决定：中心准不准（site-map）、阈值对不对（thresholds）。探测本身没有可调旋钮去“补救”标定的错误——这也是为什么前两步要做好。

10.5.3 一致性约束: reducer 与 radius 必须匹配

`reducer` 和 `radius` 在**训练阈值**和**探测时必须一致**。道理很直接: 阈值是在某种 `reducer / radius` 下的计数尺度上定出来的。如果训练时用 `mean`、 3×3 , 探测时却用 `sum`、 5×5 , 计数的绝对尺度完全不同, 那条阈值就毫无意义, 判定会系统性出错。

代码用一个干净的设计杜绝了这种错配: `reducer` 和 `radius` 被**存进** `TrapCalibration` (`calibration.reducer` 等)。当你通过 `TrapCalibration.detect` 路径探测时, 它**自动复用标定里存的 reducer/radius**, 所以“训练-探测不一致”这条路在 `TrapCalibration.detect` 上**根本走不到**。你只需在最初做 `site-map / thresholds` 时想清楚 `reducer` 和 `radius`, 之后探测会自动沿用。

10.6 第四步 (可选): detection_time 扫描成像时间

10.6.1 调用与返回

Python

```
res = exp.readout.detection_time(
    times=[5e-6, 2e-5, 1e-4], # 要扫的成像曝光时间 (秒)
    shots=60,                  # 每个时间点采多少发来估保真度
    reference_shots=30,         # 建参考阈值用多少发
    live=True,                  # 边扫边出结果
    display=True,
)
print(res.times)               # 实际扫过的时间数组
print(res.fidelities)          # 对应每个时间的保真度
res.stop()                     # 提前停止 live 扫描
```

返回 `DetectionTimeScanResult`: `res.times` 与 `res.fidelities` 一一对应; `res.stop()` 用于中途停止 live 采集。

10.6.2 原理: 用保真度曲线挑成像时间

成像时间越长, 原子荧光积累越多、亮峰越亮、暗亮越分得开、保真度越高; 但太长会带来散射加热、原子丢失、占空比下降。所以要找一个**保真度开始饱和 (plateau)** 的最短时间——再长也换不来明显收益。

扫描做法:

1. 先在一个**长曝光**下采参考帧 (启发式取 $3 * \max(\text{times})$, 确保参考足够亮、分得很开), 由参考帧建一条**参考阈值**。
2. 对 `times` 里的每个时间, 采 `shots` 发, 用 `otsu_threshold` 在这个时间下重新定阈值, 再用 `estimate_threshold_fidelity` 估这个时间的保真度。
3. 把 `(time, fidelity)` 连成曲线。

读图方法: 在 `fidelities` 对 `times` 的曲线上, 找保真度**爬到平台**的那个拐点时间, 取拐点附近 (略偏右一点, 留余量) 作为正式成像时间。比这更长只是浪费占空比、徒增加热。

10.7 保存、加载、清除与当前标定

`exp.readout.save(path)` 把当前 `TrapCalibration` (`centers + thresholds + reducer + radius` 等) 落盘, 返回写入路径。每次实验开机标定一次后存盘, 第二天可直接加载, 省去重标。

exp.readout.load(path) 从盘上读回一份标定并设为当前。返回该 `TrapCalibration`。

exp.readout.clear() 清空当前标定 (把会话上的标定置空)。之后必须重新 site-map / thresholds 才能 detect。

exp.readout.current 只读属性, 返回当前生效的 `TrapCalibration`, 没有则为 `None`。脚本里探测前可用它确认“标定确实在位且带阈值”。

加载后仍要 sanity-check

加载的标定是按当时的相机视场和阵列位置定的。如果光路漂了、相机动了、阱阵列重排了, 旧 centers 会偏。加载后建议先 `detect(display=True)` 看一眼亮环是否还套在真实亮斑上, 再投入正式采集。

10.8 一次完整的端到端示例

Python

```
# 0) 假设 exp 是已建好的会话, 相机、光阱已就绪、阵列尽量装满

# 1) site-map: 在满阵列上找 5x7 个中心
sm = exp.readout.sitemap(frames=20, grid_shape=(5, 7),
                        ordering="row-major", roi_radius=1,
                        reducer="mean", display=True)
print(" 找到中心数: ", len(sm.calibration.centers)) # 期望 35

# 2) thresholds: 建暗/亮直方图并定阈值
th = exp.readout.thresholds(frames=100, site=0, display=True)
print("0 号位点保真度: ", th.fidelity)
print(" 阈值范围: ", th.thresholds.min(), "~", th.thresholds.max())

# 3) (可选) 扫成像时间, 挑保真度饱和点
dt = exp.readout.detection_time(times=[5e-6, 2e-5, 1e-4],
                                shots=60, reference_shots=30,
                                live=True, display=True)
for t, f in zip(dt.times, dt.fidelities):
    print(f"t={t:g}s -> fidelity={f:.4f}")

# 4) 存盘, 方便下次直接 load
exp.readout.save("calib_today.json")

# 5) 逐发探测 (这一步可以在采数循环里反复调用)
det = exp.readout.detect(display=True, what="occupancy")
print(" 占据下标: ", det.detection.occupied_indices)
print(" 占据数: ", int(det.detection.occupied.sum()))
```

10.9 常见失败与排查

site-map 找的中心数不对 / 圆环没套在亮斑上 多半是 `grid_shape` 写错 (行列反了或总数不符)、阵列没装满 (违反“全占据”假设)、或亮度太弱触发了“取最亮 N 像素”兜底分支。对策: 核对 `grid_shape=(ny, nx)` 与真实排布一致; 确保标定时阵列尽量全满、均匀; 加大 `frames` 提信噪比; 看 `res.plot` 用肉眼复核。

两个相邻阱被找成一个 / 一个阱被找成两个 阱间距和 `min_distance`、平滑 `sigma` 不匹配。间距很密

时局部极大邻域可能把两个阱并成一个峰；噪声大时一个斑可能裂成两峰。对策：保证标定图够干净（多平均），必要时检查 `grid_shape` 是否与实际密度相符。

thresholds 的直方图不是双峰 / 阈值落在奇怪的地方 暗亮分不开。常见原因：成像时间太短（亮峰不够亮、和暗峰糊在一起）、背景过高（暗峰右移）、或某些阱在标定期间根本没原子。对策：先用 `detection_time` 找到保真度饱和的成像时间再定阈值；检查 `res.counts` 直方图确认是否真双峰；压低杂散光。

保真度偏低（明显小于 0.99） 暗亮高斯重叠大。看 `FidelityEstimate` 里的 `separation`：偏小说明两峰太近，需要更长成像时间或更低背景。也注意 `left_fraction/right_fraction` 是否严重失衡（某一类样本太少），这会触发置信度修正把保真度往 0.5 拉，提示“数据本身不足以下结论”。

detect 结果系统性偏多/偏少占据 几乎一定是**阈值整体偏了**或**reducer/radius 不一致**。本系统通过把 `reducer/radius` 存进 `TrapCalibration` 并在 `TrapCalibration.detect` 里自动复用，从设计上杜绝了不一致这条路。阈值偏了则回到 `thresholds` 重标（成像条件变了就必须重标）。

detect 报“center 越界”或“ROI 无有限像素” `roi_counts` 会校验中心在图像范围内、ROI 里有有限像素。报错通常意味着加载的旧标定与当前相机视场不匹配（视场/裁剪变了），或图像里有 NaN/Inf。对策：确认相机 ROI/分辨率与标定时一致；必要时 `clear()` 后重新 `site-map`。

加载旧标定后判定全错 光路或相机漂移导致 `centers` 偏离真实亮斑。对策：加载后先 `detect(display=True)` 目视确认，发现偏了就重做 `site-map`（漂移大时阈值也要重标）。

换了 ordering 导致下标对不上 同一会话里前后用了不同 `ordering`，`occupied` 向量的下标含义就变了。对策：在一套实验里**固定一种 ordering**，并在分析脚本里用同一约定解读位点编号。

排查总原则

读出链是“`site-map` → `thresholds` → `detect`”的单向依赖。判定出问题时，**从上游往下游查**：先用 `res.plot` 确认中心对不对，再看 `res.counts` 直方图确认阈值合不合理，最后才怀疑 `detect` 本身——而 `detect` 是纯阈值比较，几乎从不“自己出错”，错都在它上游的标定里。

11

标定结果对象与完整实验流程（深入）

本章面向第一次接触本系统的读者，目标是把“如何把一张相机原始图像变成『每个阱位有没有原子』的判定”以及“如何把这套判定嵌进一次完整的实验采集循环”讲透。我们会先把核心数据对象 `TrapCalibration` 逐字段拆开，再讲结果对象（result objects）及其关键不变量——**覆盖层永不改写原始图像数组**，然后给出可直接运行的整套 notebook 采集流程、扫描配合读出的循环，最后是一整节排错清单。

阅读本章前，你只需要知道：相机会拍一张二维图像；图像上有一组规则排布的光阱（trap），每个阱位中心对应图像里的一个小区域（ROI）；我们要在每个 ROI 里统计亮度，再用阈值判断该阱位是否被原子占据。`TrapCalibration` 就是承载“中心坐标 + 阈值 + ROI 几何 + 统计方式”这一整套标定信息的对象。

11.1 TrapCalibration 深入：每个字段的含义

`TrapCalibration` 定义在 `core/calibration.py`。它是一个纯数据 + 行为的标定对象：既保存做点位探测（detection）所需的全部参数，又自带把图像变成判定结果的方法。把它理解成“一份可序列化、可复现的探测配方”。

11.1.1 字段清单

centers 形状为 `(N, 2)` 的数组，表示 `N` 个阱位中心在图像中的像素坐标。这是标定的核心：每个中心配合 `roi_radius` 框出一个用于统计亮度的 ROI。`N` 即阱位总数，可由 `n_sites()` 读出。约定每行是一个 `(row, col)` 或 `(y, x)` 像素坐标对，务必与你后续画覆盖层、切 ROI 的取轴方式保持一致。

thresholds 占据判定阈值。它**既可以是形状为 `(N,)` 的逐阱位阈值数组，也可以是一个标量**。当它是 `(N,)` 数组时，每个阱位用自己的阈值（适合阱位间亮度不均匀的真实系统）；当它是标量时，所有阱位共用同一阈值（适合刚搭起来、还没逐位标定的快速起步阶段）。统计量超过阈值即判为“有原子”。

grid_shape 阱位的二维排布形状 `(ny, nx)`，或 `None`。若阵列本身是规则的二维网格（例如 `(5, 5)` 共 25 个阱位），设置它能让结果对象以二维方式聚合与作图（例如把占据率画成 `(ny, nx)` 的热图）；若阱位排布不规则，或你只关心一维序列，置 `None` 即可。注意 `ny*nx` 应与 `N` 一致。

roi_radius ROI 半径，整数（像素）。它决定以每个 center 为圆心 / 方心切出多大的区域来做亮度统计。

半径太小会丢光、信噪比差；太大则会把相邻阱位的光或背景纳入，造成串扰。它是探测灵敏度的关键旋钮之一。

reducer ROI 内像素的**归约方式**（把一片像素压成一个标量），取值为 `"mean"`、`"sum"`、`"median"`、`"max"` 之一。`"sum"` 对总光子数敏感、随 ROI 面积变化；`"mean"` 与面积无关、更稳健；`"median"` 抗热点 / 坏像素；`"max"` 对单点亮斑敏感。**reducer 一旦选定，阈值就是相对这个统计量定的——换 reducer 必须重新定阈值，否则判定全错（见排错节）。**

metadata 字典，记录标定的附带信息与来源标记。一个典型条目是 `\{"thresholds_calibrated": True\}`，表示这组阈值是真正测过的、而非占位的标量默认值。metadata 不参与探测计算，但对追溯“这份标定是怎么来的、能不能信”至关重要。

一句话记住四个旋钮

`centers` 决定**在哪测**，`roi_radius` 决定**测多大一块**，`reducer` 决定**怎么把一块压成一个数**，`thresholds` 决定**这个数多大算有原子**。四者环环相扣，改任一个通常都要回头校验另外三个。

11.1.2 方法清单

`n_sites()` 返回阱位数 N ，等于 `centers` 的行数。用于循环、分配结果数组、断言形状一致。

`detect(image)` 核心方法。输入一张二维图像，**用对象内已存的 `centers` / `thresholds` / `roi_radius` / `reducer`** 对每个阱位切 ROI、归约、与阈值比较，返回该帧的探测结果。它不接受额外探测参数——所有参数都来自对象本身，这正是“标定即配方”的体现：同一个 `TrapCalibration` 作用在任意图像上，行为完全确定、可复现。

`with_thresholds(thresholds, **metadata)` 返回一个**新的** `TrapCalibration`，其阈值被替换为传入的 `thresholds`，并可通过关键字参数合并进 metadata（例如 `with_thresholds(th, thresholds_calibrated=True)`）。它不就地修改原对象，便于在保留原始标定的同时派生出“刚标好阈值”的新版本。

`to_dict()` / `from_dict()` 在对象与普通 Python 字典之间互转。`to_dict()` 把所有字段序列化成可 JSON 化的结构；`from_dict(d)` 反向重建对象。它们是 `save/load` 的底层，也方便你手动检视或拼装标定。

`save(path)` 把标定写盘。**格式由扩展名决定**：`.json` 写成人可读的 JSON（便于 diff、手改、版本控制审阅）；`.npz` 写成紧凑的 NumPy 压缩包（体积小、读写快，适合大 N 或频繁存取）。

`load(path)` 从盘读回，按扩展名自动选择解析方式，返回 `TrapCalibration`。

11.1.3 schema 与存盘格式注意事项

- **选.json 还是.npz**：要让人读、要进 git、要手动核对，用 `.json`；要省空间、 N 很大、当作中间产物频繁存取，用 `.npz`。两者承载相同字段，可互换重建。
- `thresholds` 的多态性（标量 vs `(N,)` 数组）会被原样保存与恢复。读回后若要做逐元素运算，记得先判断它是标量还是数组。
- `grid_shape` 为 `None` 时表示“无网格语义”，序列化后仍是 `None`；不要用 `(N, 1)` 之类去冒充，那会触发二维聚合路径。
- `metadata` 里务必让来源标记诚实：阈值真标过就置 `thresholds_calibrated=True`，否则留 `False` 或不设。下游和排错都依赖这个标记判断标定是否可信。

Python

```

from core.calibration import TrapCalibration
import numpy as np

# 从零构造一份标定: 5x5 网格, 先用一个标量阈值起步
centers = np.array([[y, x] for y in range(5) for x in range(5)],
    ↪ dtype=float)
calib = TrapCalibration(
    centers=centers,          # (25, 2)
    thresholds=1200.0,       # 标量: 所有阱位共用, 起步用
    grid_shape=(5, 5),       # 规则网格 -> 结果可二维聚合
    roi_radius=3,            # 以每个 center 切 7x7 区域
    reducer="sum",           # 统计 ROI 内总计数
    metadata={"thresholds_calibrated": False}, # 诚实标记: 还没真正标阈值
)

print(calib.n_sites())      # 25

# 存成人类可读 JSON, 便于审阅 / 进版本库
calib.save("calib_startup.json")

# 读回完全等价的对象
calib2 = TrapCalibration.load("calib_startup.json")

```

Python

```

# 真正标好逐阱位阈值之后, 派生一份新标定 (不改原对象)
per_site_th = np.full(calib.n_sites(), 1200.0) # 实际由 thresholds
    ↪ 标定流程得到
calib_final = calib.with_thresholds(
    per_site_th,
    thresholds_calibrated=True, # metadata 同步置真
)
calib_final.save("calib_final.npz") # 紧凑格式存盘

```

11.2 结果对象与“覆盖层不改原图”不变量

实验过程中每一步都会产生一个**结果对象** (result object), 定义在 `core/results.py`。它们都是 notebook 友好的: 自带 `.plot` 用于即看即画, 并会**自动追加进** `exp.history`, 从而支持事后回放 (replay) 与导出。

11.2.1 结果对象家族

CaptureResult 一次相机采集的产物, 承载原始帧及其元信息, 是后续所有分析的起点。

SitemapResult 阱位定位 (sitemap) 的结果, 对应找出 `centers` 的那一步——把“光阱在图像哪里”固化下来。

ThresholdResult 阈值标定的结果, 对应把每个阱位的占据判定阈值定出来的那一步, 最终喂给 `TrapCalibration.thresholds`。

DetectionResult 一次探测 (detect) 的结果: 每个阱位的统计量与“有没有原子”的判定, 通常配合 `.plot` 把判定覆盖在图像上看。

DetectionTimeScanResult 随时间的探测扫描结果，对应 `detection_time(live=True)` 这类持续观测——把占据 / 计数随帧（时间）的变化聚成序列。

11.2.2 关键不变量：覆盖层永不改写原始图像

所有可视化里，**阱位圆圈等覆盖层（overlay）**都是画在 `matplotlib` 的坐标轴上，绝不修改底层的原始图像数组。这是一条必须牢记的不变量，原因有三：

1. **原始数据始终可信**：你看到的圈只是注释层，底下的像素永远是相机给的原值。任何复算、重标定、重判定都基于未被污染的数据，结果可复现。
2. **可反复重画而不累积副作用**：换 `roi_radius`、换 reducer、换阈值后重画覆盖层，不会因为“上次画的圈被烧进了图像”而出错。
3. **回放与导出安全**：因为 `exp.history` 里存的是干净数据 + 画法，事后从 history 重新出图、导出，得到的永远是一致的结果。

别去“擦掉”覆盖层

如果你想换一种画法，直接用结果对象的 `.plot` 重画即可——不要尝试在图像数组上“涂”或“擦”圈。覆盖层和数据是两层，分开的，这正是系统刻意保证的。

11.3 完整 notebook 实验流程

下面给出从连接设备到持续探测的标准 notebook 循环。会话入口与各子系统定义在 `session.py`。

11.3.1 会话与子系统总览

`connect(config, ...)` 返回一个 `NeutralAtomSession`（习惯命名为 `exp`）。它把硬件与逻辑分解成若干子系统：

`exp.devices` `DeviceSet`，底层设备集合。

`exp.camera` 相机句柄。

`exp.timing` `TimingSubsystem`，时序子系统。关键方法：`configure_imaging(exposure, load, trigger_width, pre_trigger)` 返回一条 `PulseSequence`；`preflight()` 返回 `PreflightReport`（其 `.raise_if_failed()` 会在时序不自洽时抛错）；`bind_pulse(pulse)` 返回 `PulseController`；`write_verilog(output_dir)` 导出 HDL。

`exp.readout` `ReadoutSubsystem`，读出子系统，封装 `sitemap` / `thresholds` / `save` / `detect` / `detection_time` 等高层动作。

`exp.sequence` 当前的 `PulseSequence`。

`exp.history` 结果对象列表，前述所有 result 自动追加于此。

11.3.2 `configure_imaging` 的四个参数

`timing.configure_imaging(exposure, load, trigger_width, pre_trigger)` 用来把一次成像的时序编出来，返回一条可绑定、可下发的 `PulseSequence`：

exposure 相机曝光时长。它必须与相机实际的曝光设置一致——这是最常见的错配来源（见排错节）。

load 装载 / 加载阶段时长 (把原子装进阱、稳态化的时间)。

trigger_width 触发脉冲宽度, 给相机的硬件触发信号有多宽。

pre_trigger 触发前导时间, 控制触发相对其他时序事件的提前量, 保证相机在曝光窗开始前已就绪。

11.3.3 标准采集循环 (可运行)

典型 notebook 流程是: **连接** → **配置成像时序** → **预检** → **定位阱位** → **标定阈值** → **存标定** → **探测** → **持续探测**。

Python

```
from session import connect

# 1) 连接, 拿到会话对象 exp
exp = connect(config)

# 2) 配置成像时序, 得到一条 PulseSequence
pulse = exp.timing.configure_imaging(
    exposure=20e-3,      # 与相机实际曝光一致!
    load=100e-3,
    trigger_width=1e-6,
    pre_trigger=5e-6,
)

# 3) 预检: 时序不自洽就当场抛错, 绝不带病往下跑
exp.timing.preflight().raise_if_failed()

# 4) 定位阱位 -> SitemapResult (确定 centers)
sitemap = exp.readout.sitemap()

# 5) 标定阈值 -> ThresholdResult (确定逐阱位 thresholds)
thresholds = exp.readout.thresholds()

# 6) 存标定, 落盘以便复用 / 审阅
exp.readout.save("calib_today.json")

# 7) 探测一帧 -> DetectionResult, 画出判定覆盖层
det = exp.readout.detect()
det.plot  # notebook 里直接出图: 圈画在坐标轴上, 原图数组不动

# 8) 持续探测 (实时) -> DetectionTimeScanResult
ts = exp.readout.detection_time(live=True)
```

有限帧采集: 别让 qCMOS 等死循环

做有限帧采集时, 正确姿势是**先 arm 相机, 再精确生成“恰好需要的那么多个触发”**。切勿让 qCMOS 相机去等一个 **repeat-forever (永远重复)** 的脉冲循环——那样相机会一直等下去而你拿不到确定数量的帧。要数得清几帧, 就发几个触发。

11.3.4 每一步产出了什么

- `sitemap()` 产出 `SitemapResult`, 其核心是 `centers`。

- `thresholds()` 产出 `ThresholdResult`, 其核心是逐阱位阈值, 对应把 `TrapCalibration.thresholds` 从标量升级为 `(N,)` 数组, 并应把 `metadata` 的 `thresholds_calibrated` 置真。
- `save(path)` 把当前读出标定写盘 (`.json` 或 `.npz`, 规则同 `TrapCalibration.save`)。
- `detect()` 产出 `DetectionResult`, 每个阱位一个统计量 + 一个布尔占据判定。
- `detection_time(live=True)` 产出 `DetectionTimeScanResult`, 把占据 / 计数随时间聚成序列, 适合盯加载、盯寿命。

11.4 扫描配合读出：一次扫描一帧一判定

把脉冲扫描和读出串起来, 可以做“扫一个参数、每个扫描点拍一帧、判定一次、最后聚成 1D/2D 结果”的实验。要点是**硬件无缝迭代扫描点, 每个扫描点恰好一个相机触发**。

流程是：用 `timing.bind_pulse(pulse)` 得到 `PulseController`, 配上一张扫描表 (`scan table`); 时序器会**无缝地 (seamlessly) 逐点迭代**扫描表, **每个扫描点产生一个相机触发**; 每帧用标定去 `detect`; 再把每个扫描点的占据 / 计数聚合成一维或二维结果。

每点一帧 扫描表有多少个点, 就有多少个触发、多少帧、多少次判定——一一对应。这正是有限帧采集“发几个触发拿几帧”原则在扫描场景下的体现。

无缝迭代 扫描点由时序器在硬件中连续推进, 不需要主机在点与点之间反复重配, 保证时序确定、节奏稳定。

聚合维度 若扫描是一维 (扫一个参数), 结果聚成 1D (占据率 / 计数随该参数); 若与 `grid_shape` 或二维扫描配合, 可聚成 2D。

Python

```
# 绑定脉冲, 拿到可下发扫描的控制器
ctrl = exp.timing.bind_pulse(pulse)

# 提供一张扫描表 (N_points 个扫描点; 具体构造见时序/脉冲章节)
# ctrl 会无缝迭代每个扫描点, 每点发一个相机触发
occupancy = []
for frame in run_scan_with_one_trigger_per_point(ctrl, scan_table):
    det = exp.readout.detect(frame)    # 用已存标定判定这一帧
    occupancy.append(det)              # 收集每点的判定

# 把逐点判定聚合成 1D/2D 占据 / 计数结果, 画出来
# (聚合后的结果对象同样会进入 exp.history, 可回放 / 导出)
```

别让相机干等

扫描循环里同样遵守**先 arm 相机、再发恰好 N_points 个触发**的原则。绝不要把相机挂在 `repeat-forever` 的脉冲循环后面等帧——扫描点是有限的, 触发数必须精确等于扫描点数。

11.5 排错清单

本节集中处理三类最常踩的坑：预检失败、曝光错配、标定过期。

11.5.1 预检 (preflight) 失败

`exp.timing.preflight()` 返回 `PreflightReport`, 调用 `.raise_if_failed()` 会在时序不自洽时抛异常。**务必在采集前调用并让它抛错**, 而不是跳过——它正是用来在你浪费一整轮采集之前, 把时序问题挡在门外。

- 看报告抛出的具体原因: 通常是 `configure_imaging` 的参数组合不自洽 (例如 `pre_trigger` 或 `trigger_width` 与 `exposure/load` 的时间窗冲突)。
- 修正 `configure_imaging` 的入参后**重新生成 pulse 并再次 preflight**, 直到通过。
- 不要用 `try/except` 吞掉 `raise_if_failed()` 的异常然后继续——那等于关掉了安全网。

11.5.2 曝光错配 (exposure mismatch)

`configure_imaging` 里的 `exposure` 必须与**相机端实际的曝光设置一致**。两边不一致是经典 bug:

- 症状: 图像偏亮 / 偏暗, 统计量整体偏移, 导致用旧阈值判定时占据率异常 (全有或全无)。
- 根因: 时序按一个曝光排、相机按另一个曝光拍, 光积分时间对不上。
- 处理: 核对相机配置里的曝光与传给 `configure_imaging` 的 `exposure`, 统一为同一值; 改了曝光后**阈值通常需要重标** (因为统计量随曝光整体缩放)。

11.5.3 换相机后标定过期 (calibration staleness)

标定是和具体相机、具体成像条件强绑定的。**一旦换了相机** (或显著改了光路、增益、曝光), 原来的 `TrapCalibration` 就可能过期:

- `centers` 可能失效: 不同相机的像素映射 / 视场 / 取向不同, 阱位在图像里的位置会变, 旧 `centers` 切到的 ROI 可能根本不对准阱位。
- `thresholds` 必然需要重标: 增益 / 量子效率 / 噪声不同, 统计量的绝对尺度变了, 旧阈值的判定不再可信。
- `reducer` 与 `roi_radius` 的合适取值也可能随新相机的像素尺度变化——别假设照搬就行。
- **用 metadata 自查**: 检查 `metadata["thresholds_calibrated"]`。若为 `False`, 说明阈值还是占位值, 不可用于正式判定; 换相机后应把它视为 `False`, 重新走 `sitemap` → `thresholds` → `save`。

换相机后的标准复位动作

换相机后请按顺序重做: `configure_imaging` (用新相机的真实曝光) → `preflight().raise_if_failed()` → `readout.sitemap()` (重定 `centers`) → `readout.thresholds()` (重标阈值, `metadata` 置真) → `readout.save()` (覆盖旧标定)。跳过任何一步都可能让后续判定静默地错。

11.5.4 快速自检表

1. 采集前是否调用了 `preflight().raise_if_failed()` 且通过?
2. `configure_imaging` 的 `exposure` 是否等于相机实际曝光?

-
3. `thresholds` 是标量还是 `(N,)` 数组? `metadata["thresholds_calibrated"]` 是否为真?
 4. 改过 `reducer` 或 `roi_radius` 后, 阈值是否重标过?
 5. 有限帧 / 扫描采集是否做到了“先 arm、再发恰好需要的触发数”, 而非让 qCMOS 等 `repeat-forever` 循环?
 6. 换过相机 / 光路后, 是否重跑了 `sitemap` → `thresholds` → `save`?

12

修改指南

12.1 我要加一台自定义设备

实现对应的类型契约 (`CameraDevice/SequencerDevice/TrapArrayDevice` 之一) , `na.register_device_class("Name", cls)` 注册, 然后在配置 JSON 里用 `type: "Name"` 引用。设备只负责硬件动作; 标定、检测算法属于 `core/`, 不要塞进设备类。

12.2 我要换标定的 reducer 或网格顺序

`reducer` 和 `ordering` 决定了计数与格点编号; **阈值是针对它们学的**。换了就要重新 `sitemap + thresholds`, 别让训练用一个 `reducer`、检测用另一个。建议把 `reducer/ordering` 记进 `TrapCalibration.metadata` 并在检测时校验。

12.3 我要改本手册

正文模板在 `Zou_lab_control/neutral_atom/content/manual_templates/device_manual_zh.texbody`, 构建入口是 `build_device_manual()` (`Zou_lab_control/neutral_atom/notes.py`)。本手册用内嵌 TikZ 图, 无外部图依赖, 改完直接重建 PDF。